

스트레스 예측을 위한 안면 이미지 데이터 분석 프로젝트

팀장 윤수영

팀원 박성호 허호준 정준영 김건우 송지미 이승희 이지영

목차

1. 팀원 소개

2. 기획 의도

3. 개발 일정

4. 사용기술 · 개발도구

5. DB 설계

6. 플로우차트

7. 주요 기술

8. 팀 구현

팀원소개



윤수영 / 팀장

머신러닝/딥러닝

- KoBert를 이용한 감정인식 모델 구현
- 전처리 : KoBert를 사용한 부사 증강
- Google Cloud Vision과 나이브베이즈를 사용한 스팸단어 검출 모델 구현
- 전처리 : KoNLPy의 Okt를 사용한 한국어 문장 토큰화

SERVICES

- Spring Boot, Vue, Axios를 이용한 마이다이어리 감정 인식 서비스 구현
- TTS, STT 기능을 사용한 간단 심리 검사 서비스 구현

기술 스택

Front: Vue, axios, Text To Speech, High Chart
Back: SpringBoot, Mybatis, Oracle, Python, Django

DB PL/SQL 구현

- Oracle DB를 사용한 마이 다이어리 데이터 전송 트리거 구현

배포

- Github를 이용한 형상관리
- Git Action을 이용한 자동 배포 (Django)
- AWS를 이용한 자동 배포 (Django)

기획의도

최근 정신 건강 질환 유병률이 증가하고 있지만,
병원을 방문하지 않거나 비용부담으로 적절한 치료를 하지 못하는 환자들이 많아
일상생활에서 쉽게 사용할 수 있는 일기 서비스로 정신 건강을 관리하게 함

기획의도

문제 상황 시뮬레이션 서비스를 기획하게 된 계기

불안 상황, 가상현실로 훈련... VR 게임으로 정신과 질환 치료한다

이기상 헬스조선 기자 | 입력 2017.08.23 04:30

사회불안·공포증·중독에 효과
스마트폰 연결, 집에서도 가능

집에서 하는 VR 게임 치료가 사회불안장애 환자의 불안 척도 점수를 개선하는 데 효과가 있음을 입증했다. 2명의 사회불안장애 환자에게 VR 장치를 이용해 집에서 게임 치료를 실시하게 했다. 타인 앞에서 발표하는 상황 등을 간접 체험하는 게임이었다. 환자들의 사회불안척도 평균 점수는 2주 만에 66.27점에서 53.18점으로 크게 감소했다. 김재진 교수는 "환자가 평소 불안을 느끼는 상황을 VR 기기로 간접 체험하게 하면, 실제 비슷한 상황에 맞닥뜨렸을 때 불안감을 낮추는 효과가 있다"고 말했다.

S2 시뮬레이션을 통해 다양한 상황을 연습하여 정신 건강 관리

불안감 완화

대인 관계의 이해

정신적 준비

기획의도

HTP 검사를 이용하게 된 계기



집(House)



나무(Tree)



사람(Person)

집, 나무, 사람 그림을 그려서 내면 세계와 무의식적인 내면 심리 상태를 측정

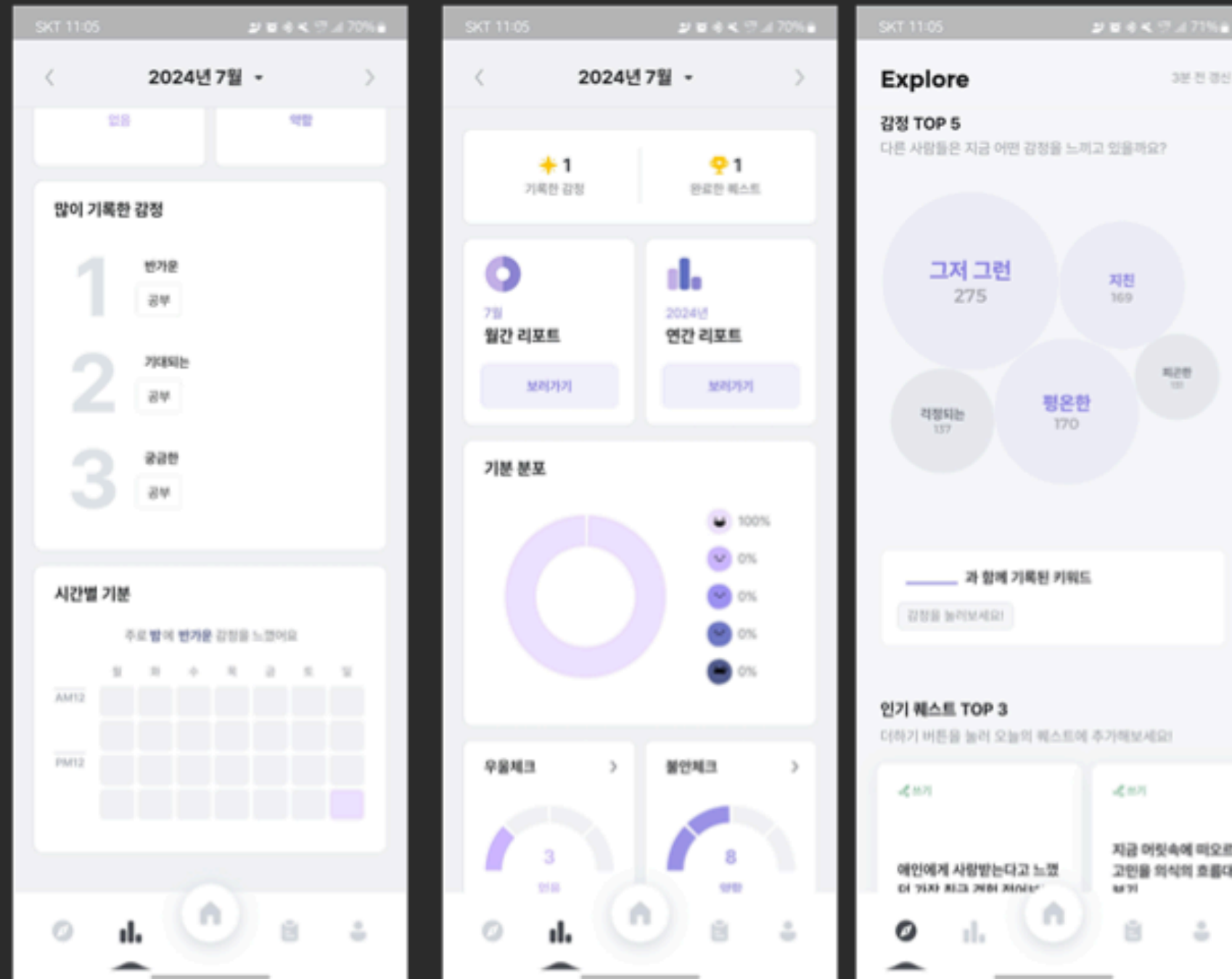
가정환경, 안정감

성장 과정, 자아상, 생명력

자아상, 대인 관계, 사회적 적응

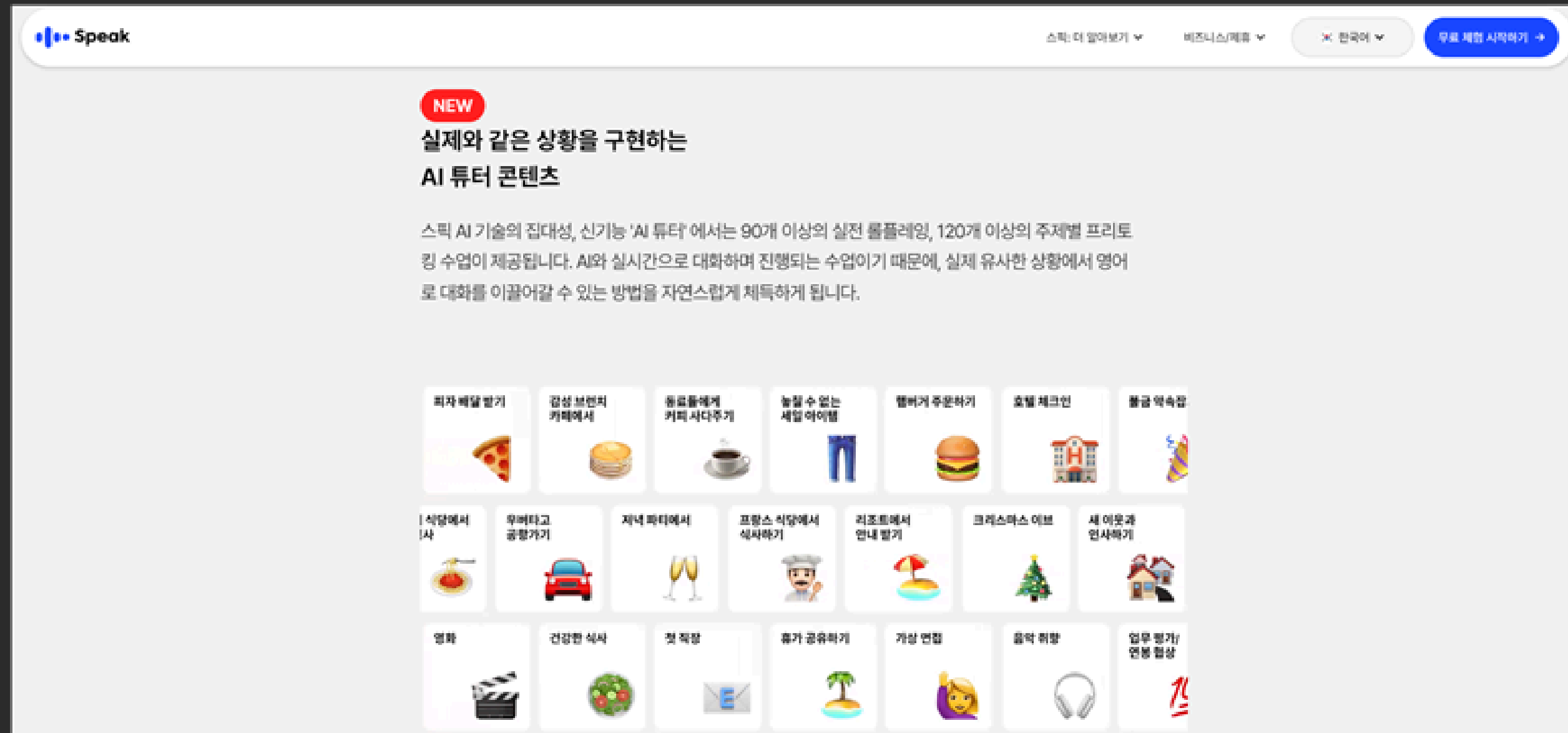
벤치마킹

벤치마킹: 무디



ai 채팅 기능을 포함한 고객의 데이터로 많은 통계 서비스를 제공하는 기능을 벤치마킹
다양한 심리 진단 검사와 시뮬레이션 서비스를 활용하여 고객의 정보를 수집하고
캘린더 형식이 아닌 감정 서비스를 이용한 후에 즉시 개인 보고서 형식으로 차별화

벤치마킹: 스픽 (시뮬레이션)



스픽의 AI 튜터 콘텐츠를 참고하여 시뮬레이션 페이지를 벤치마킹
사용자가 AI와 대화하며 실제 상황과 유사한 시뮬레이션을 경험할 수 있도록 기획

개발 일정

SUN	MON	TUE	WED	THU	FRI	SAT
					7/19	7/20
					주제 회의	
7/21	7/22	7/23	7/24	7/25	7/26	7/27
	주제회의, 개발환경 확인, 예상일정 작성, 역할분담		템플릿 선정, UI 디자인			
7/28	7/29	7/30	7/31	8/1	8/2	8/3
	ERD 제작 및 DB 구축		1차 개발 기간 : Front - Vue, Back - Spring boot 개발			
8/4	8/5	8/6	8/7	8/8	8/9	8/10
1차 개발 기간 : Front - Vue, Back - Spring boot 개발			2차 개발 기간 : DataBase와 Spring Boot 연결, 딥러닝 모델 학습			
8/11	8/12	8/13	8/14	8/15	8/16	8/17
3차 개발 기간 : 전체 코드 머지 후 오류 수정, 딥러닝 모델 Django 적용				4차 개발 기간 : 로그인 적용, API 적용		
8/18	8/19	8/20	8/21	8/22	8/23	8/24
최종 개발 기간 : 잔여 이슈 확인 및 수정, 기능 정리						
8/25	8/26	8/27	8/28			
	발표준비	발표	수료			

사용기술 · 개발도구

IDE | SERVER



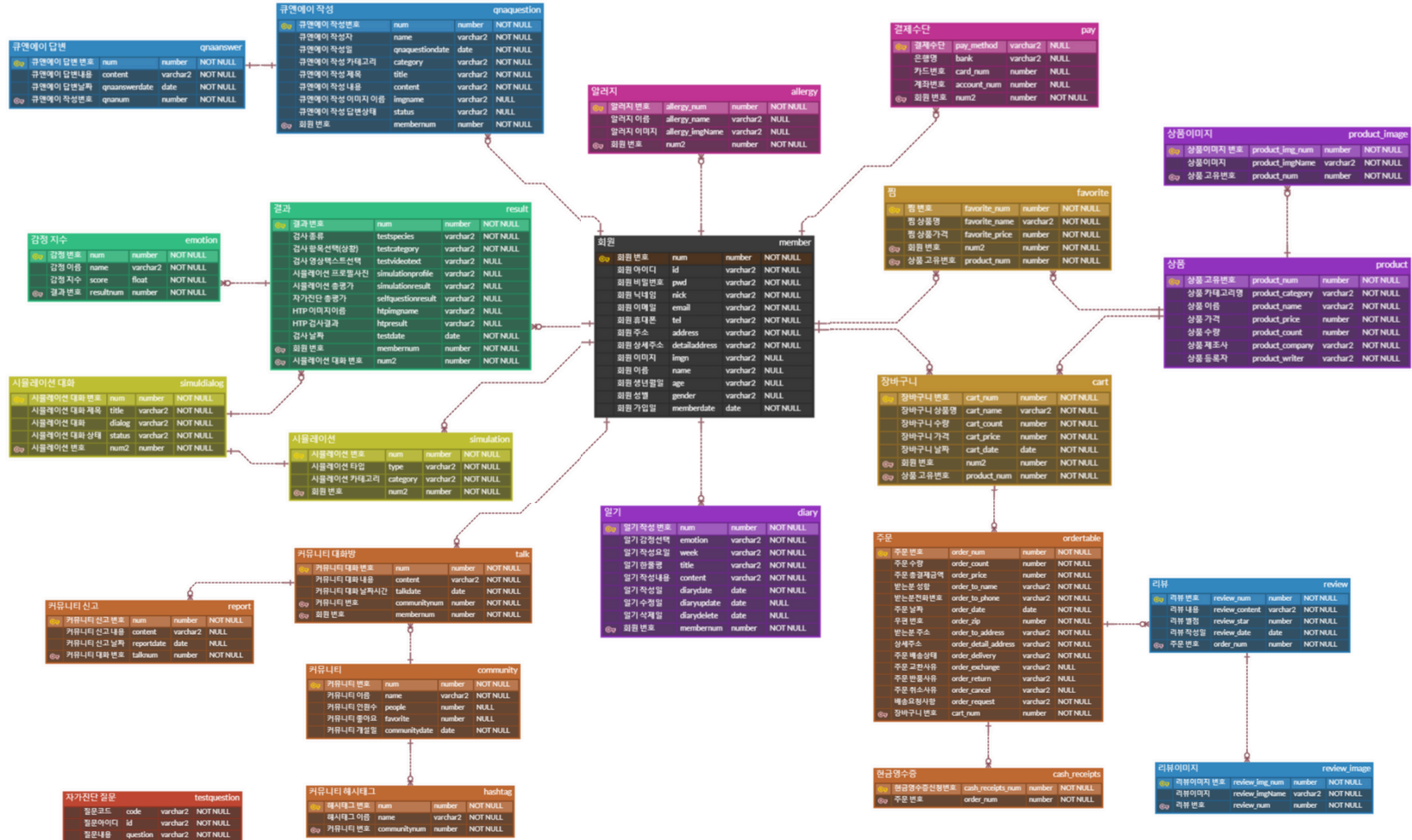
FRONT



BACK



ER 다이어그램



메뉴구성도 | S2 chat

S2 Chat 메인

S2그림 진단

s2시뮬레이션

마음나누기

Q&A상담

오늘의 마음 기록

나의마음책장

My다이어리

로그인

집 테스트 시작

시뮬레이션 고르기

채팅방입장

문의하러가기

오늘의 감정 선택

나의 마음 책

나의 감정 통계

회원가입

나무테스트시작

영상 시뮬레이션

채팅창

나의 문의목록

오늘의 일기 제목

나의 마음 수정

추천활동자세히보기

아이디찾기

남자테스트시작

텍스트 시뮬레이션

오늘의 찰칵

추천상품자세히보기

비밀번호찾기

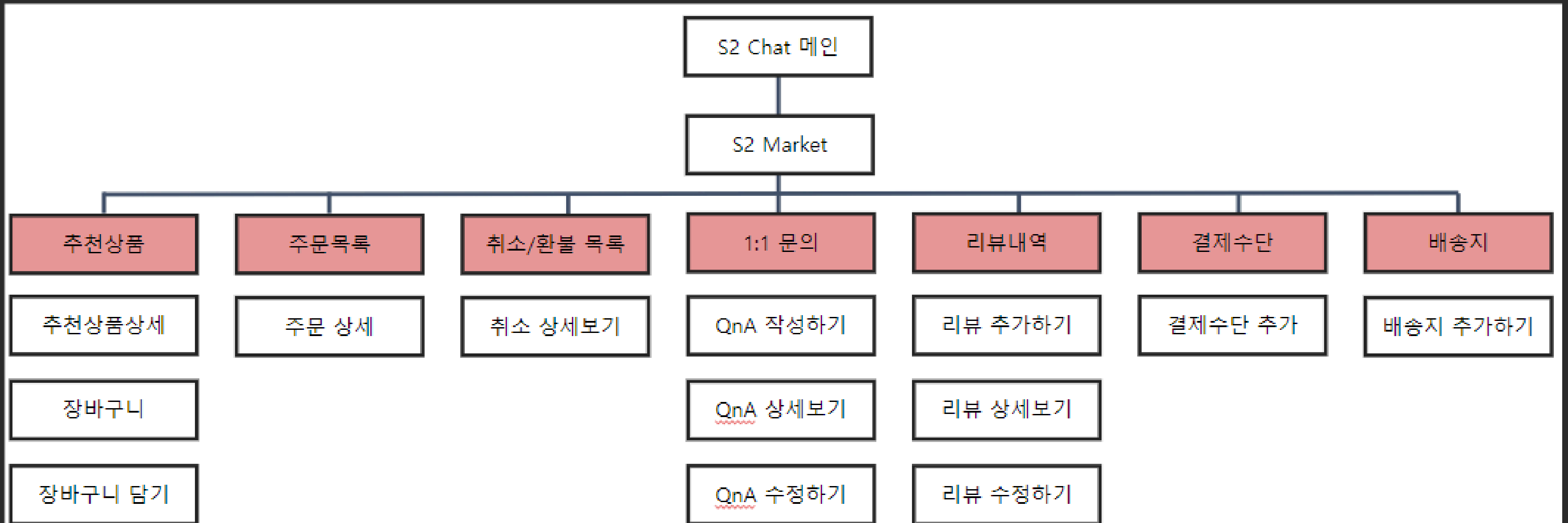
여자테스트시작

시뮬레이션 결과 보기

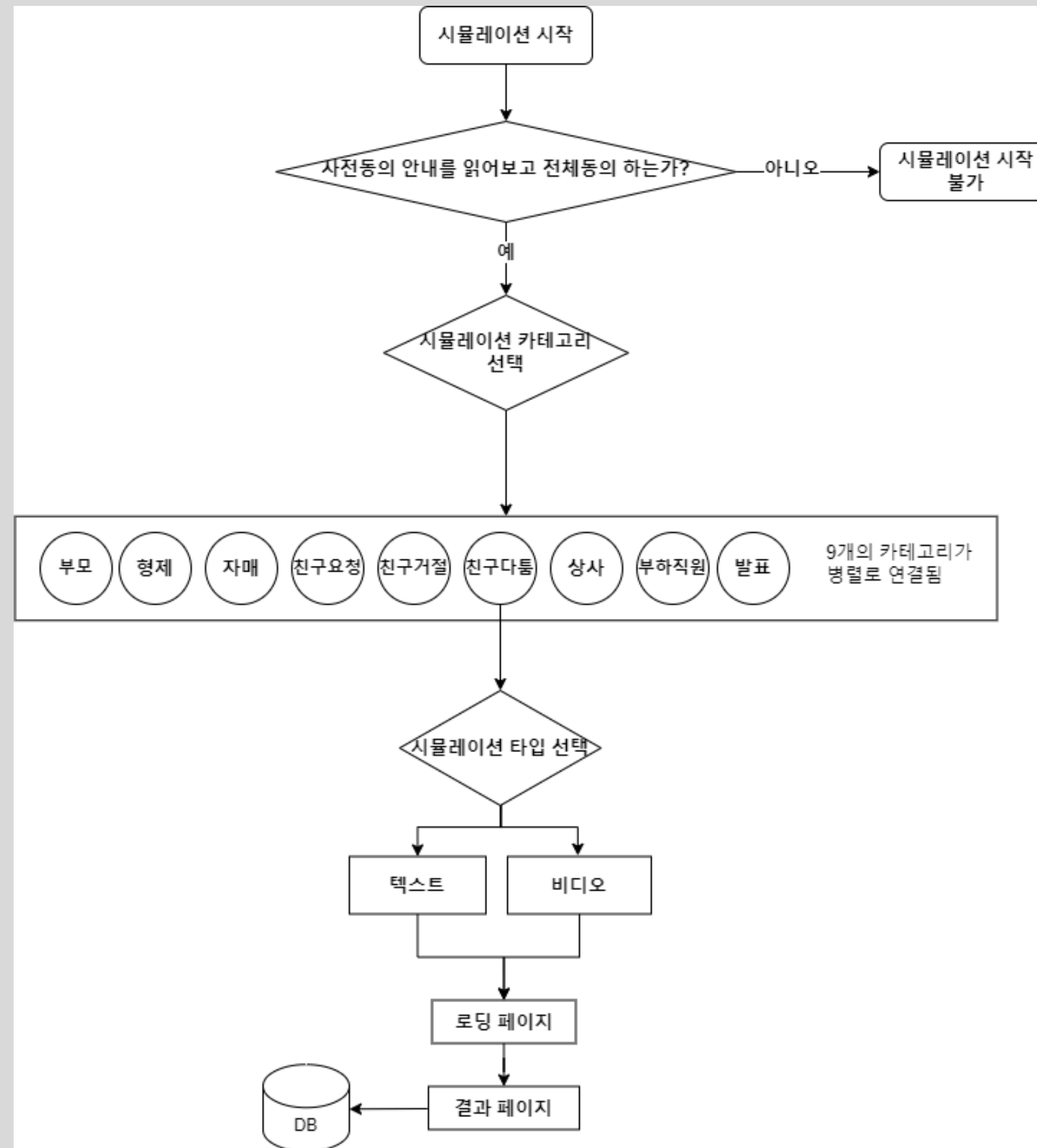
당신의 이야기

진단결과

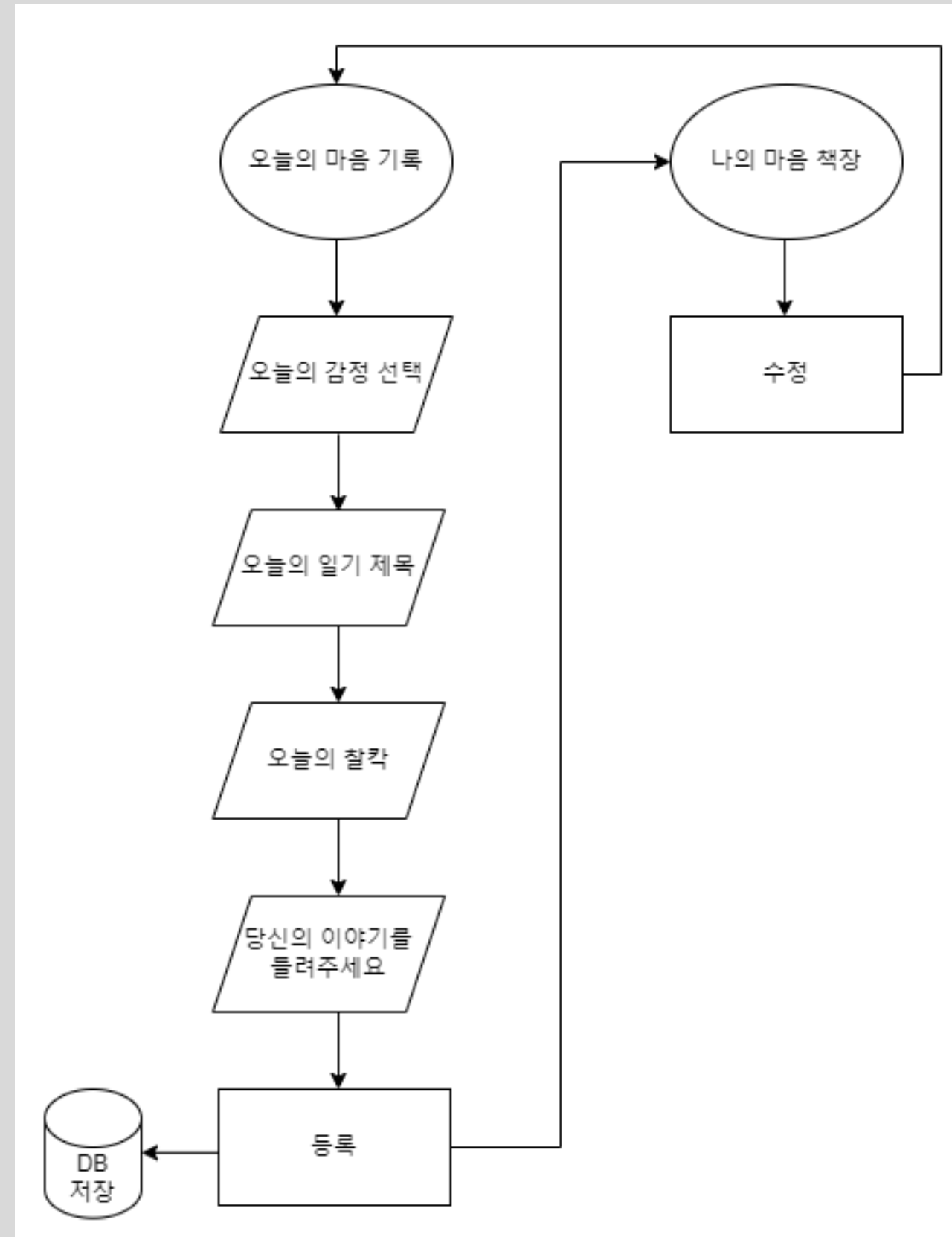
메뉴구성도 | S2 Market



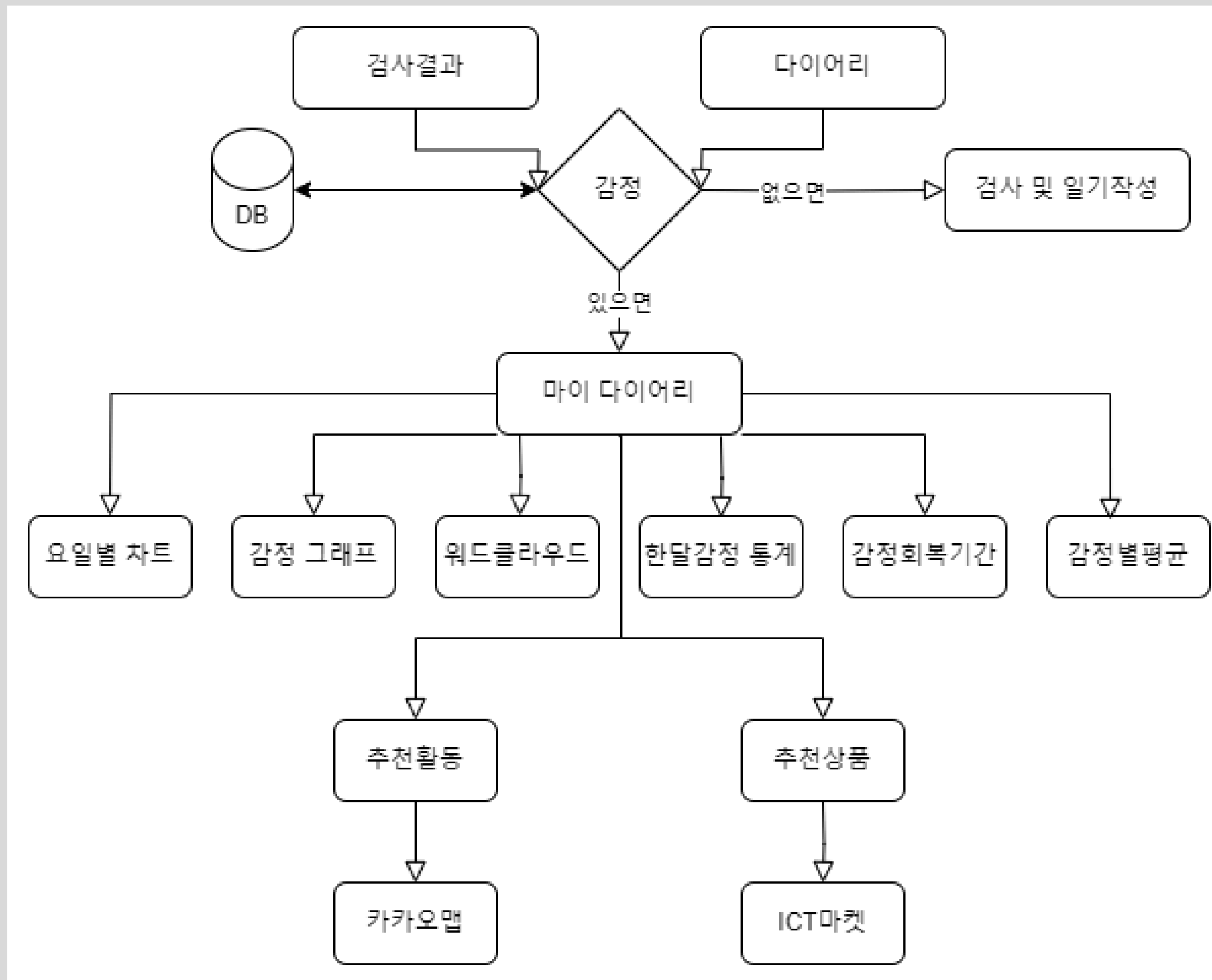
시뮬레이션 플로우차트



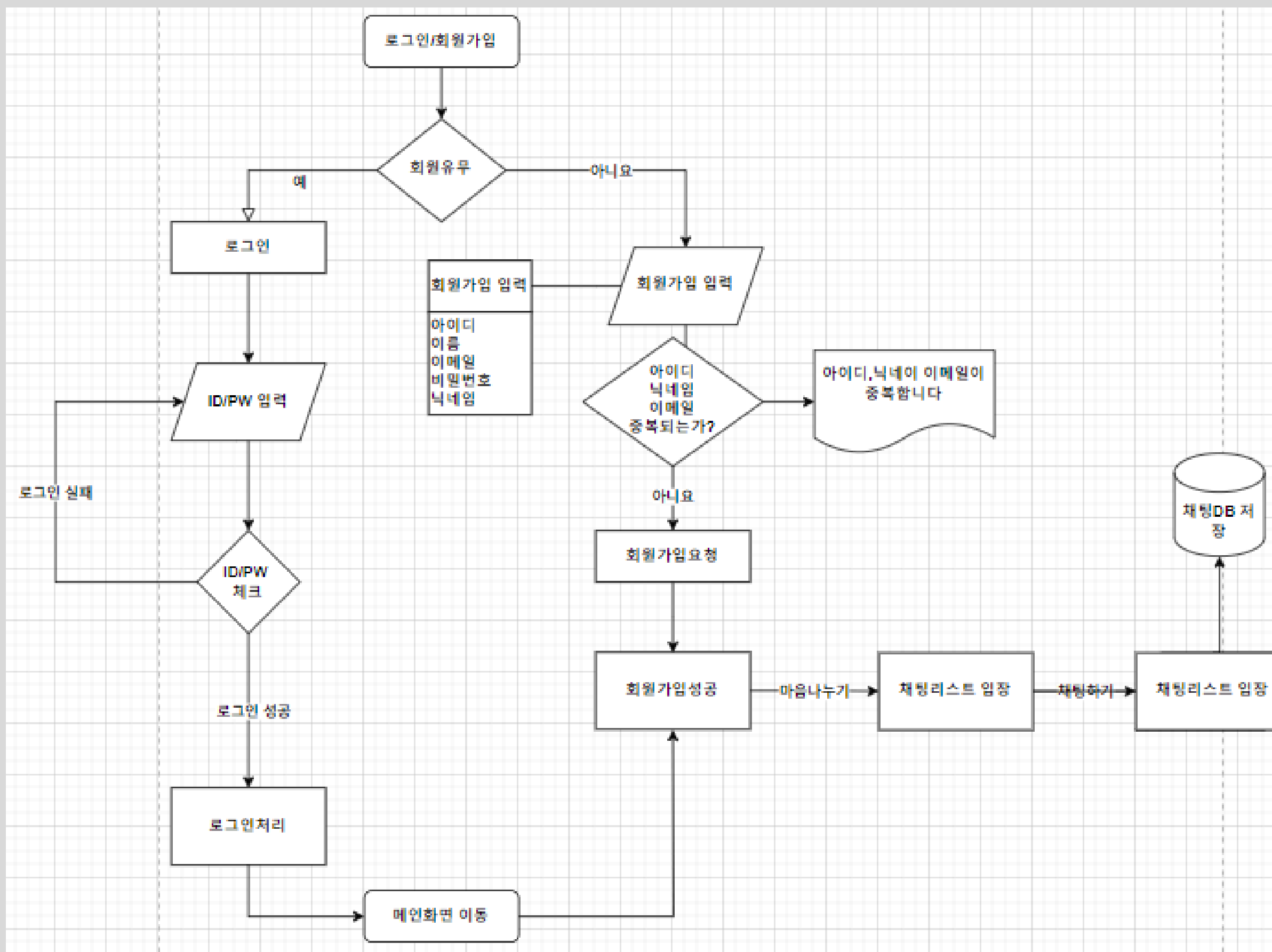
오늘의 마음 기록, 나의 마음 책장 플로우차트



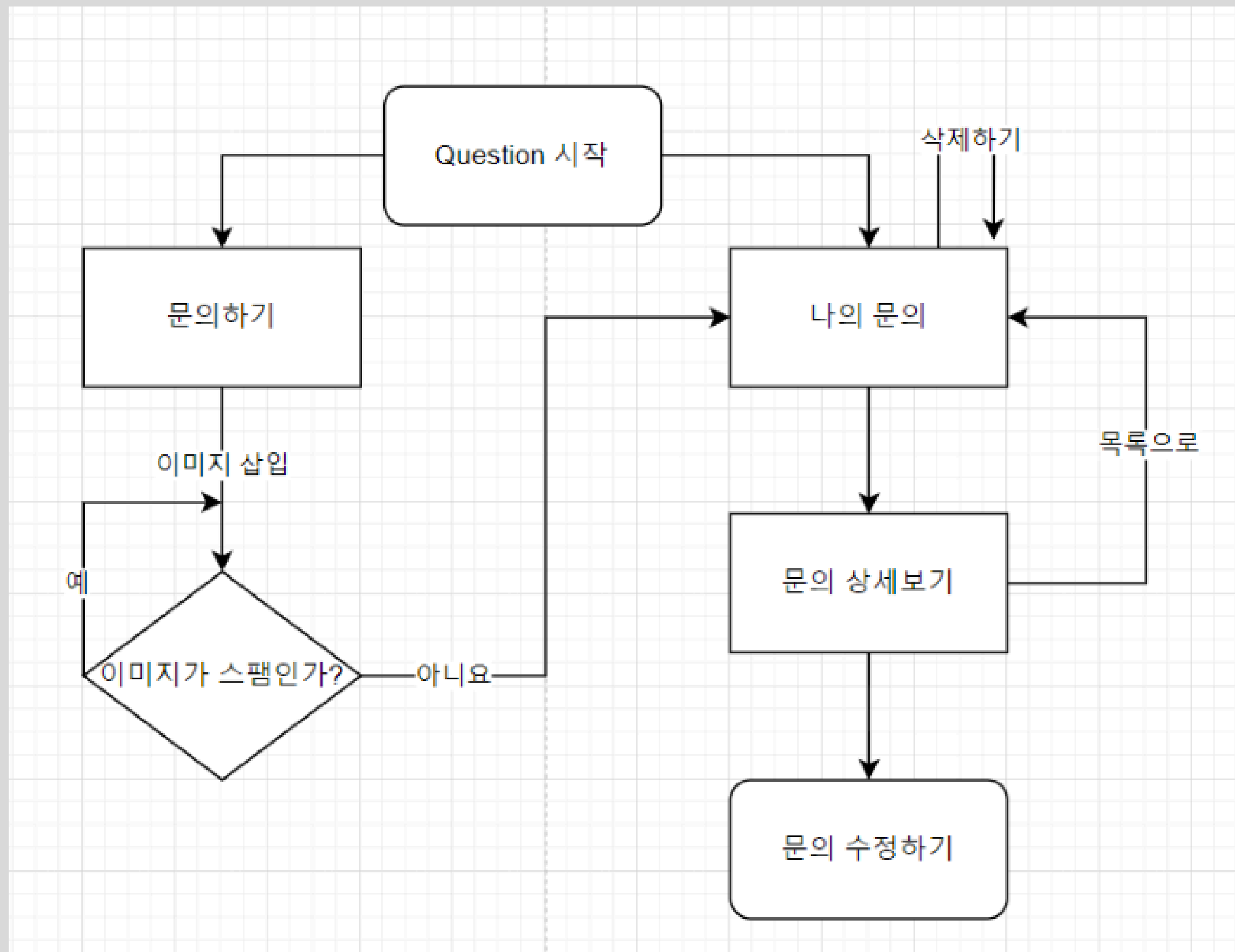
마이 다이어리 감정 통계 플로우차트



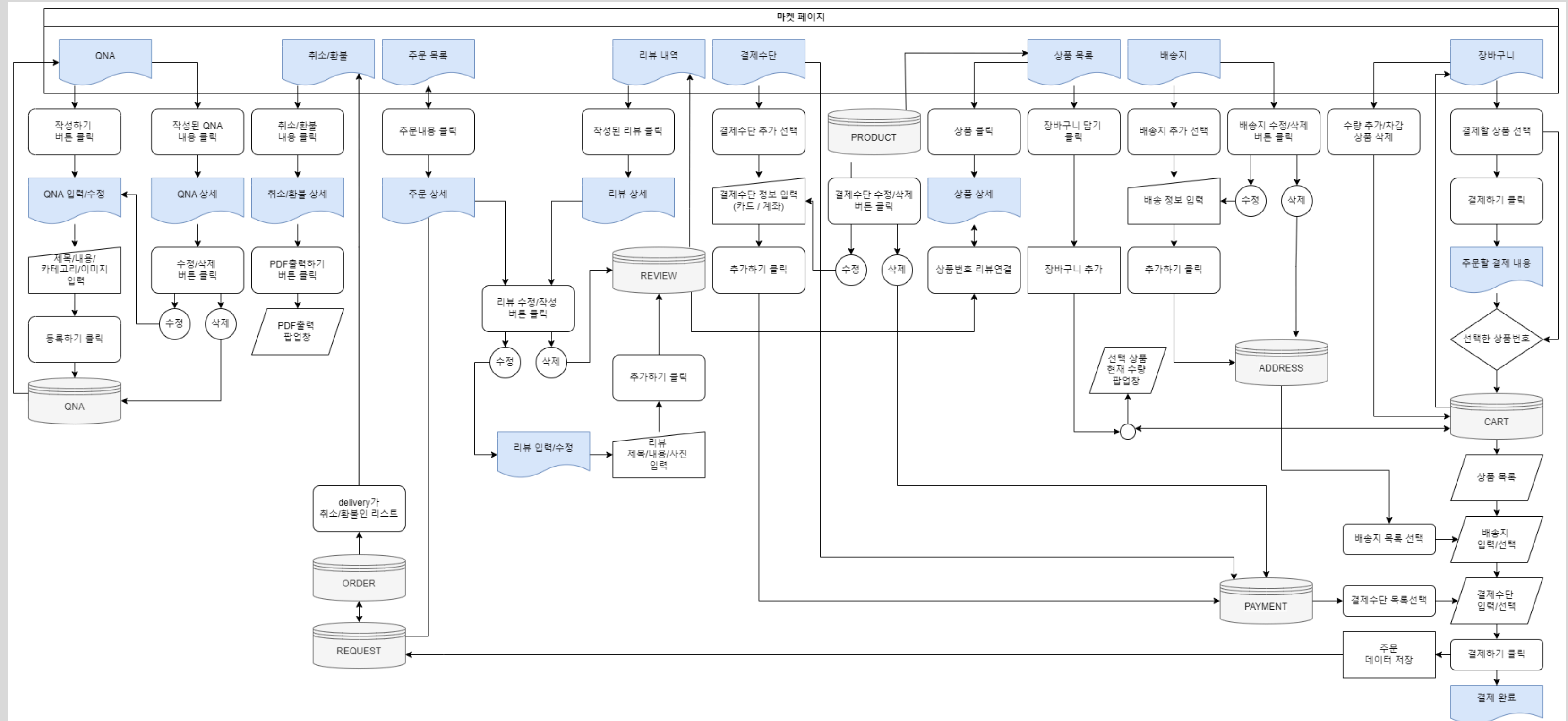
웹소켓 플로우차트



QnA 플로우차트



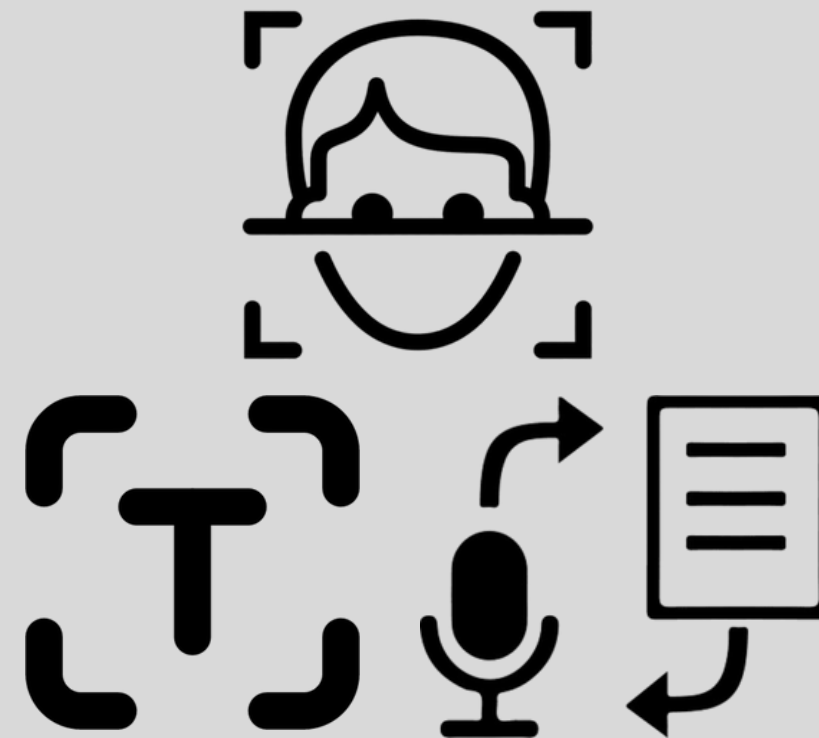
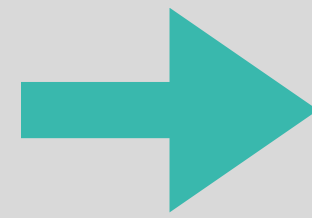
S2 Chat 쇼핑몰 페이지(구조도)



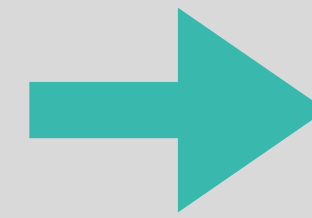
주요기능 : S2 시뮬레이션



사용자 입력



얼굴 감정인식
텍스트 감정인식
STT

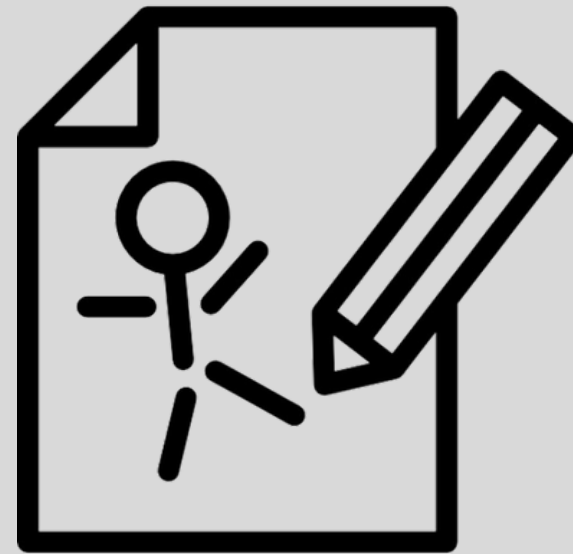
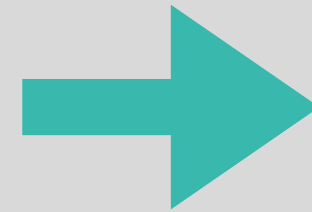


감정분석
스트레스 점수

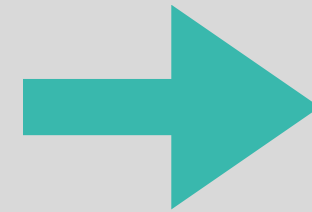
주요기능 : S2 그림 진단



그림 그리기

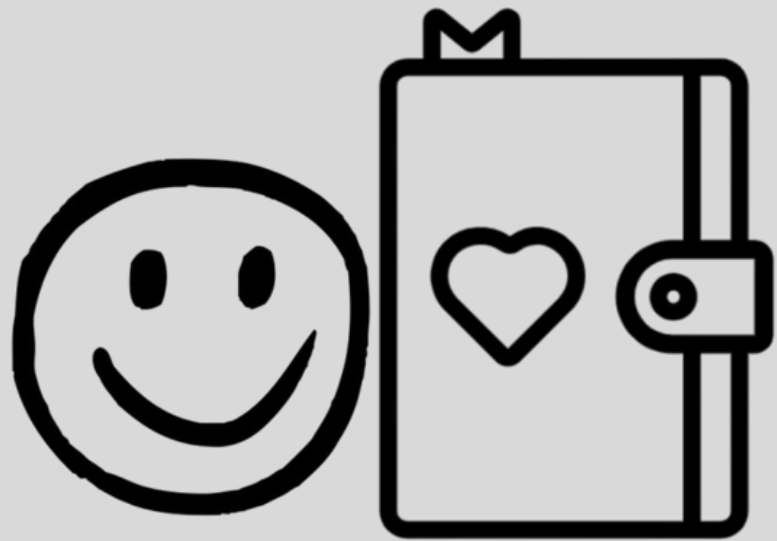


- 사용자가 제출한 그림을 분석하여 그림의 각 요소를 인식하고 분류
- 이 과정에서 그림의 특성과 의미를 추출

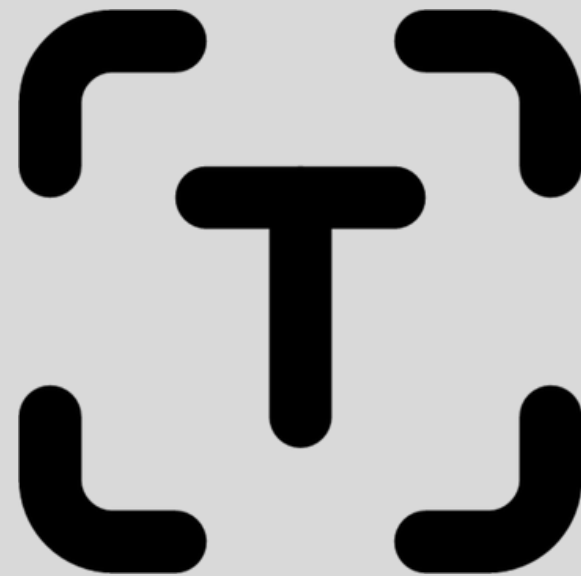
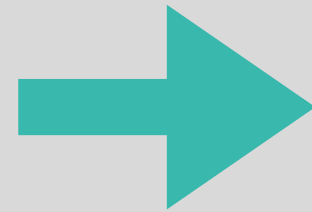


- 그림 인식 결과를 바탕으로, HTP 심리검사 이론에 따라 각 그림 요소의 심리적 의미를 해석하여 설명

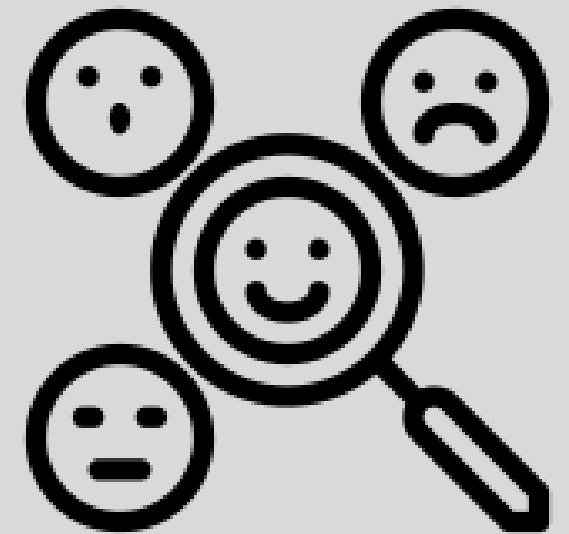
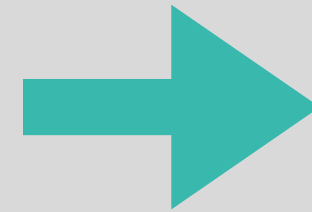
주요기능 : 마이 다이어리



사용자 입력



- 사용자가 작성한 텍스트를 분석하여 감정을 인식하고 일기에 감정적 반영 추가



- 작성된 일기를 바탕으로 감정분석

Sz Chat

02 고객상담

02 시뮬레이션

제품 나누기

Q&A 상담

오늘의 제품 기록

나의 제품 백장

MySz@에디터

실시간Market

로그인

회원가입

너와 나의
마음 돌봄 서비스

매일 당신 옆에 있어요.



192.168.0.58:3000

S2 시뮬레이션

S2 시뮬레이션 서비스 실행 화면



시뮬레이션 소개



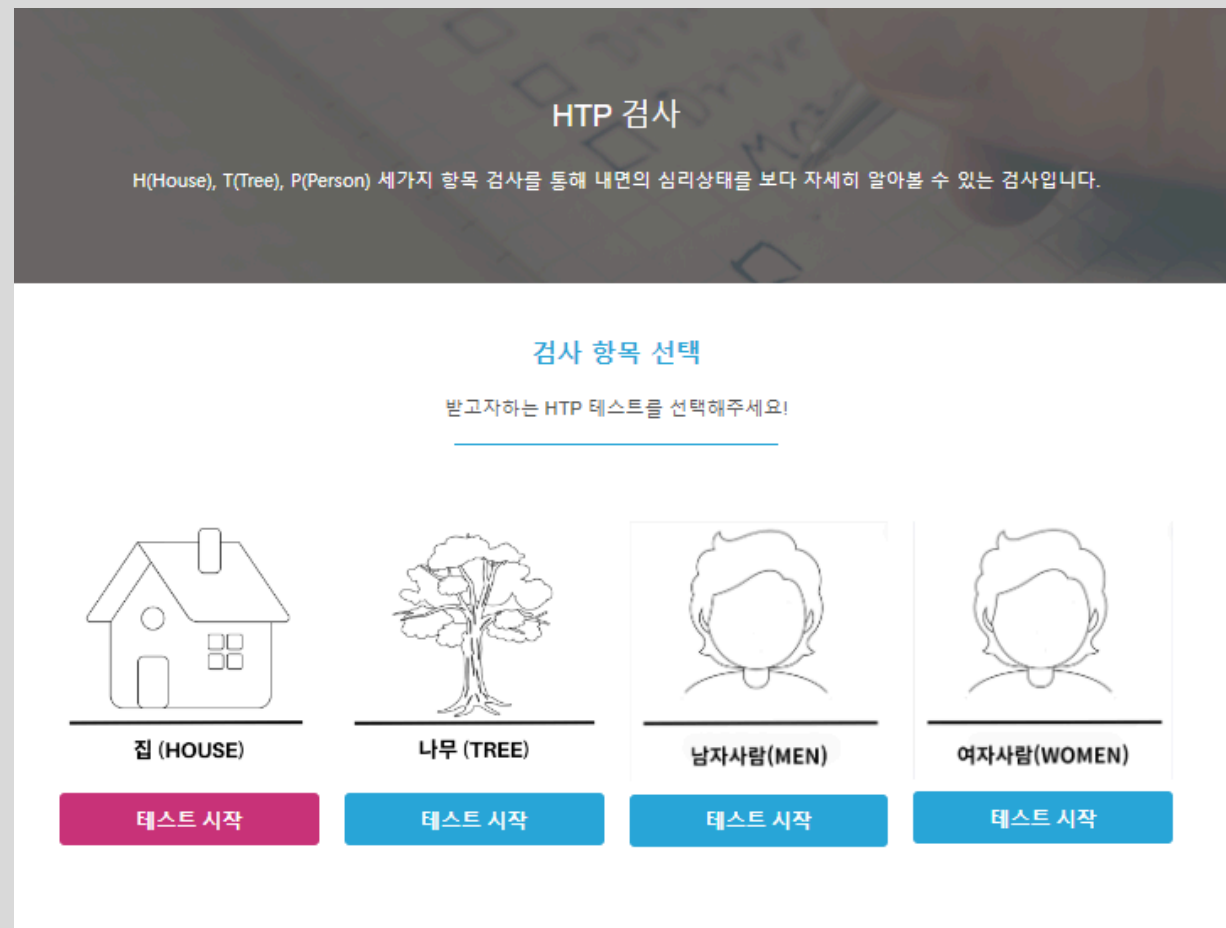
시뮬레이션 카테고리



시뮬레이션 타입

그림 심리 검사 (HTP 검사)

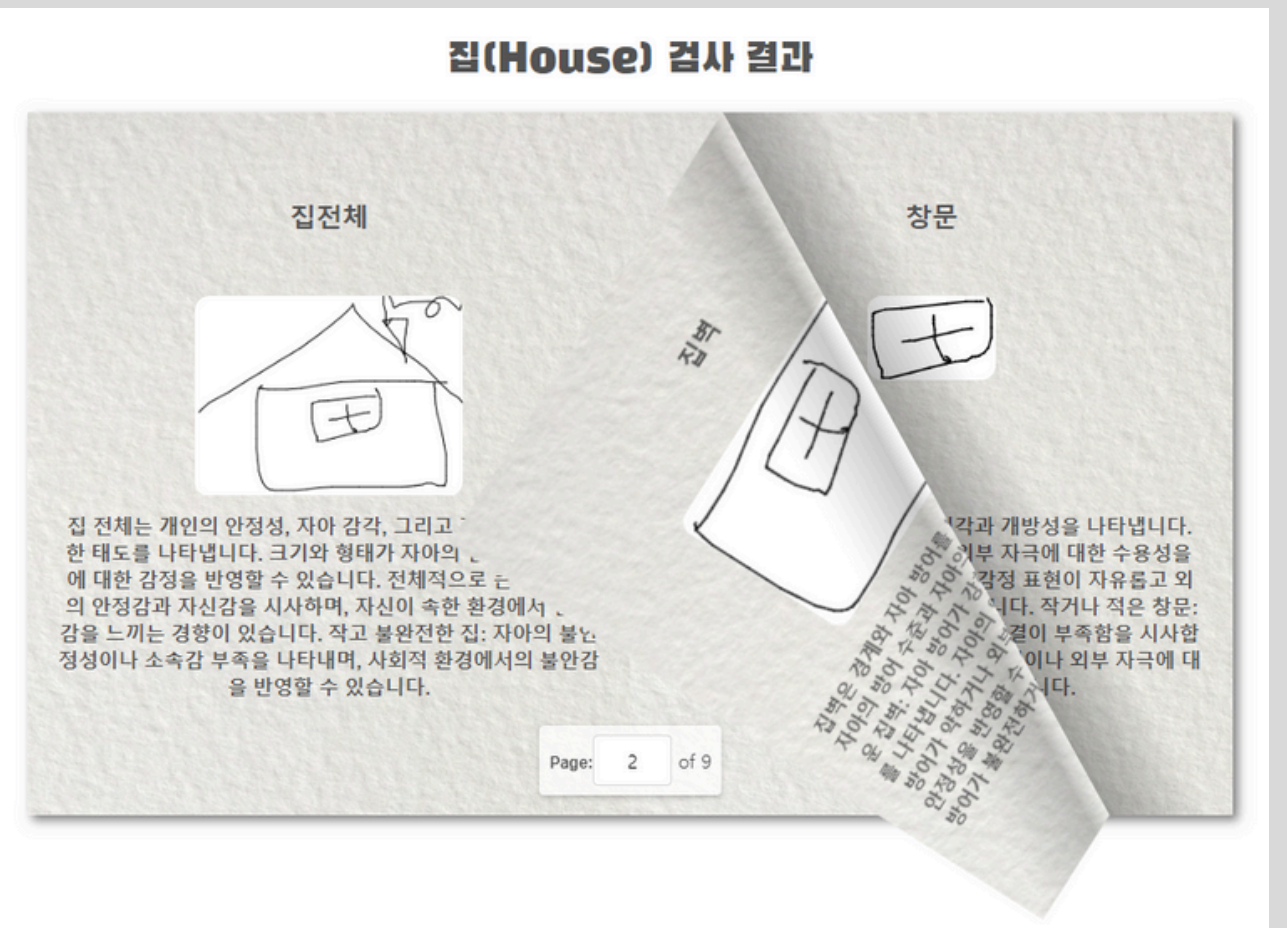
HTP 검사 서비스 실행 화면



검사 항목 선택



HTP 검사 진행



검사 결과지

마이 다이어리 | 작성

다이어리 작성화면

감정일기장

오늘의 감정 선택

행복하다

신난다

설렌다

고맙다

반갑다

부끄럽다

만족스럽다

화난다

속상하다

짜증난다

불안하다

상처받았다

감동했다

슬프다

피곤하다

지친다

후회한다

억울하다

외롭다

긴장된다

미안하다

놀랐다

걱정된다

상심하다

귀찮다

다음

오늘의 감정 선택

감정일기장

오늘의 일기 제목

날씨가 우중충해요

이전

다음

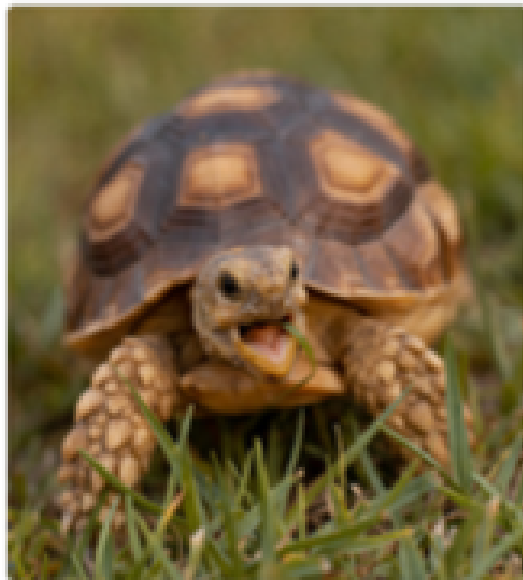
일기 제목

감정일기장

오늘의 찰칵

파일 선택

turtle.png



이전

다음

오늘의 사진

감정일기장

당신의 이야기를 들려주세요

오늘 갑자기 면접이 잡혔어요. 면접을 보러가야하는데
정장이 없어요. 그래서 갑자기 정장이 필요해 취업날개
서비스를 신청했어요. 그래서 먼저 혼자 집에 가야해요.
그렇다구요

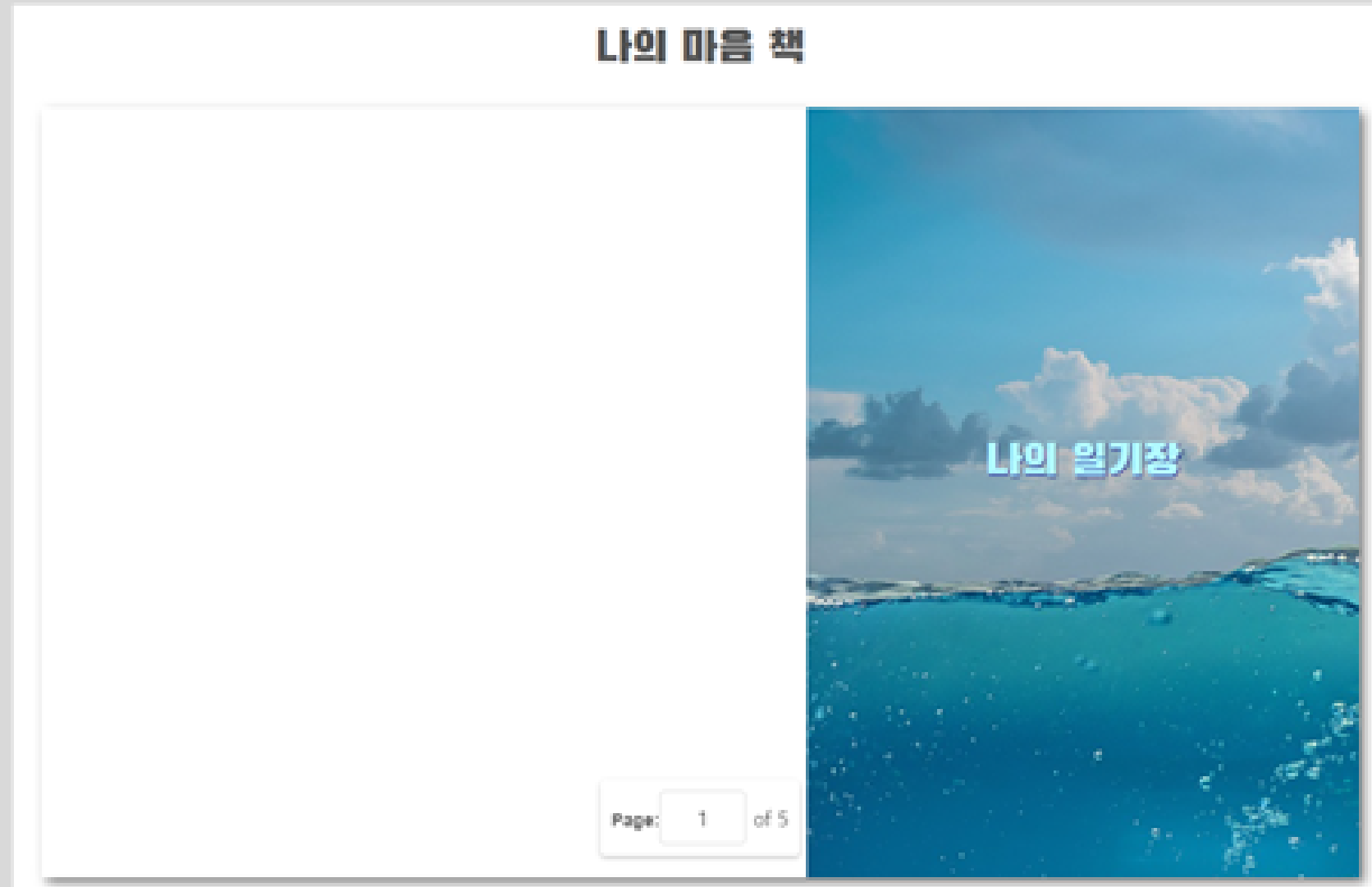
이전

저장

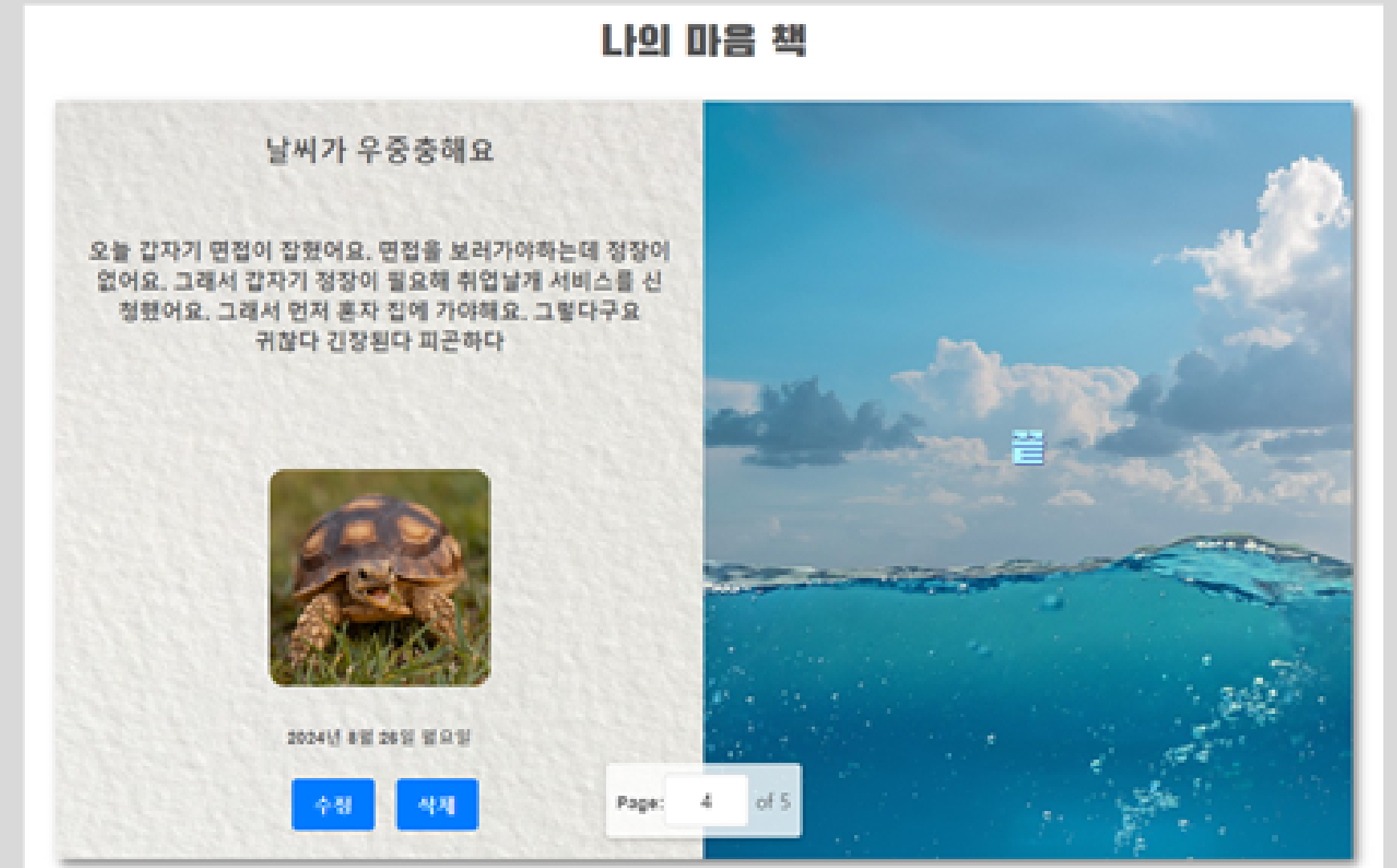
일기 쓰기

마이 다이어리 | 조회

다이어리 조회화면

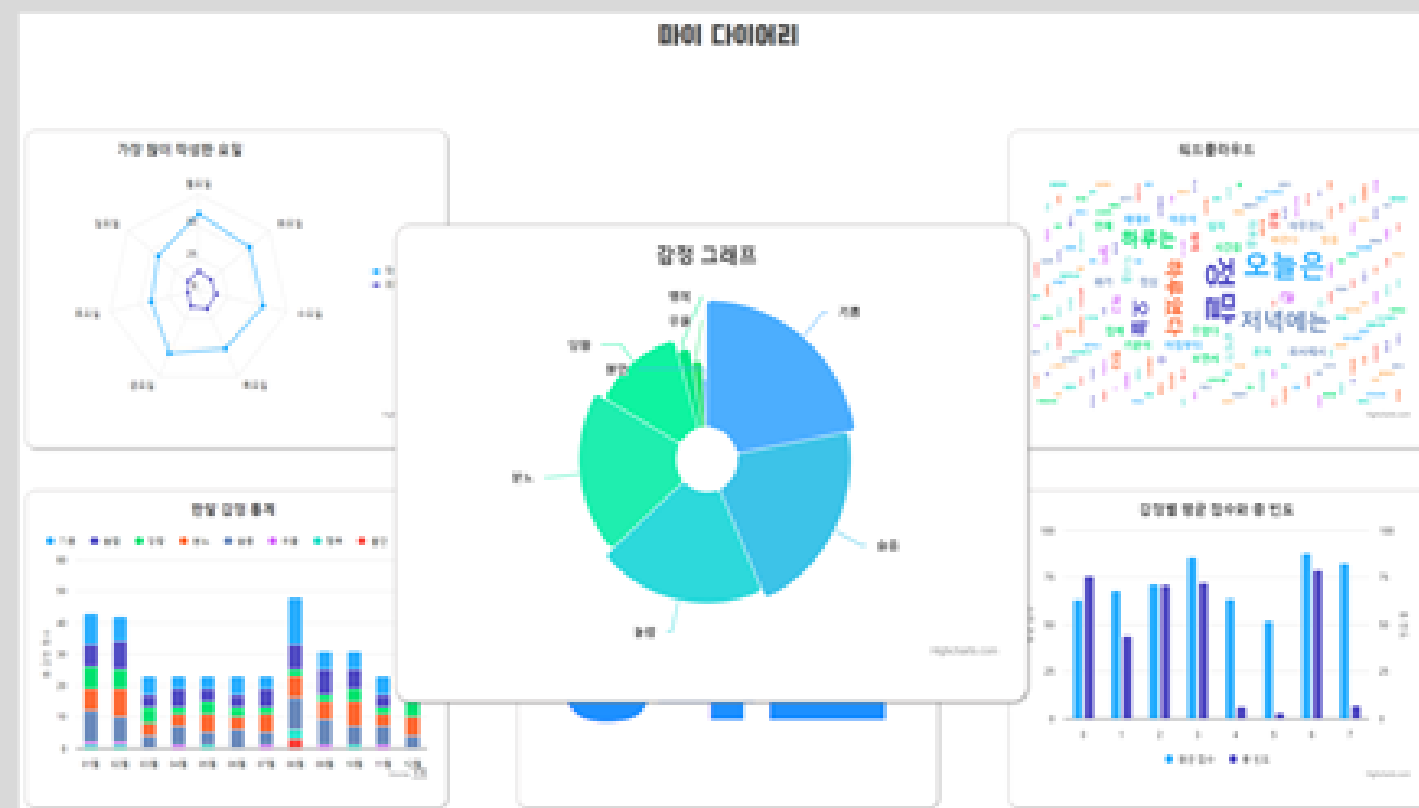
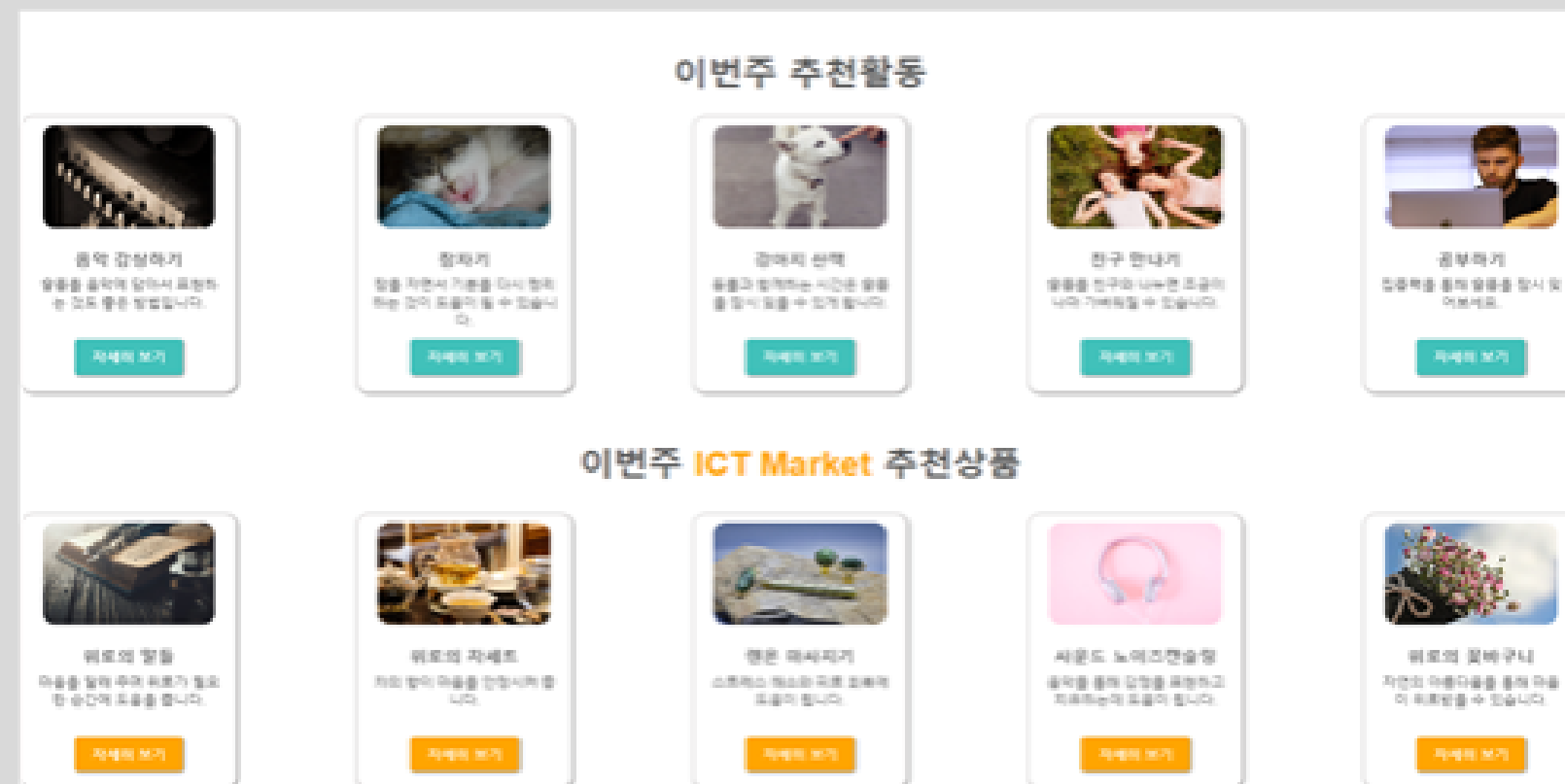


일기 표지



일기 읽기, 수정, 삭제

마이페이지 조회화면



쇼핑몰 메인화면

추천상품

주문목록

취소/환불목록

1:1문의

리뷰내역

결제수단

배송지

상표목록

전체8

진정4

불안2

무음2



~~¥~~18,900

장바구니 담기



~~19,000~~

장바구니 담기



¥12,000

장바구니 담기



#10,000

장바구니 담기



¥69,000

장바구니 담기



#23,100

장바구니 담기



¥32,000

장바구니 담기



¥19,800

장바구니 담기

S2 쇼핑몰 장바구니

상품목록 화면

심상 마켓

[추천상품](#)[주문목록](#)[취소/환불목록](#)[1:1문의](#)[리뷰내역](#)[결제수단](#)[배송지](#)

장바구니

상품목록

	상품이미지	상품명	수량	가격
☒		[티] 심신 안정에 도움을 주는 국내산 티백 세트	- 1 + 삭제	₩32,000
☐		[환] 건강한 체력을 위한 침향환 골드 선물세트	- 1 + 삭제	₩69,000
☒		[비타민] 우울한 기분을 달래줄 햇살같은 비타민 D 30일분	- 1 + 삭제	₩10,000

[결제하기](#)

CNN을 활용한 안면이미지 표정 감정 분석 모델

데이터 확인



#감정 분석 #표정 분석

한국인 감정인식을 위한 복합 영상

분야 영상이미지 유형 이미지

구축년도 : 2020 갱신년월 : 2023-10 조회수 : 32,078 다운로드 : 4,723 용량 : 835.61 GB

다운로드

↓ 샘플 데이터 ?

관심데이터 등록 155

소개

파일 목록 (API 다운로드)

카테고리(종)	이미지 개수(개)	분포(%)
기쁨	70,735	14.47%
당황	70,457	14.41%
분노	68,835	14.08%
불안	69,965	14.31%
상처	70,103	14.34%
슬픔	70,508	14.42%
중립	68,173	13.94%
평균	69,825	14.28

```
expressions = os.listdir(folder_path)

expression_counts = []
for expression in expressions:
    expression_path = os.path.join(folder_path, expression)
    num_images = len(os.listdir(expression_path))
    print(f"{expression} : {num_images} images")
    expression_counts.append(num_images)
```

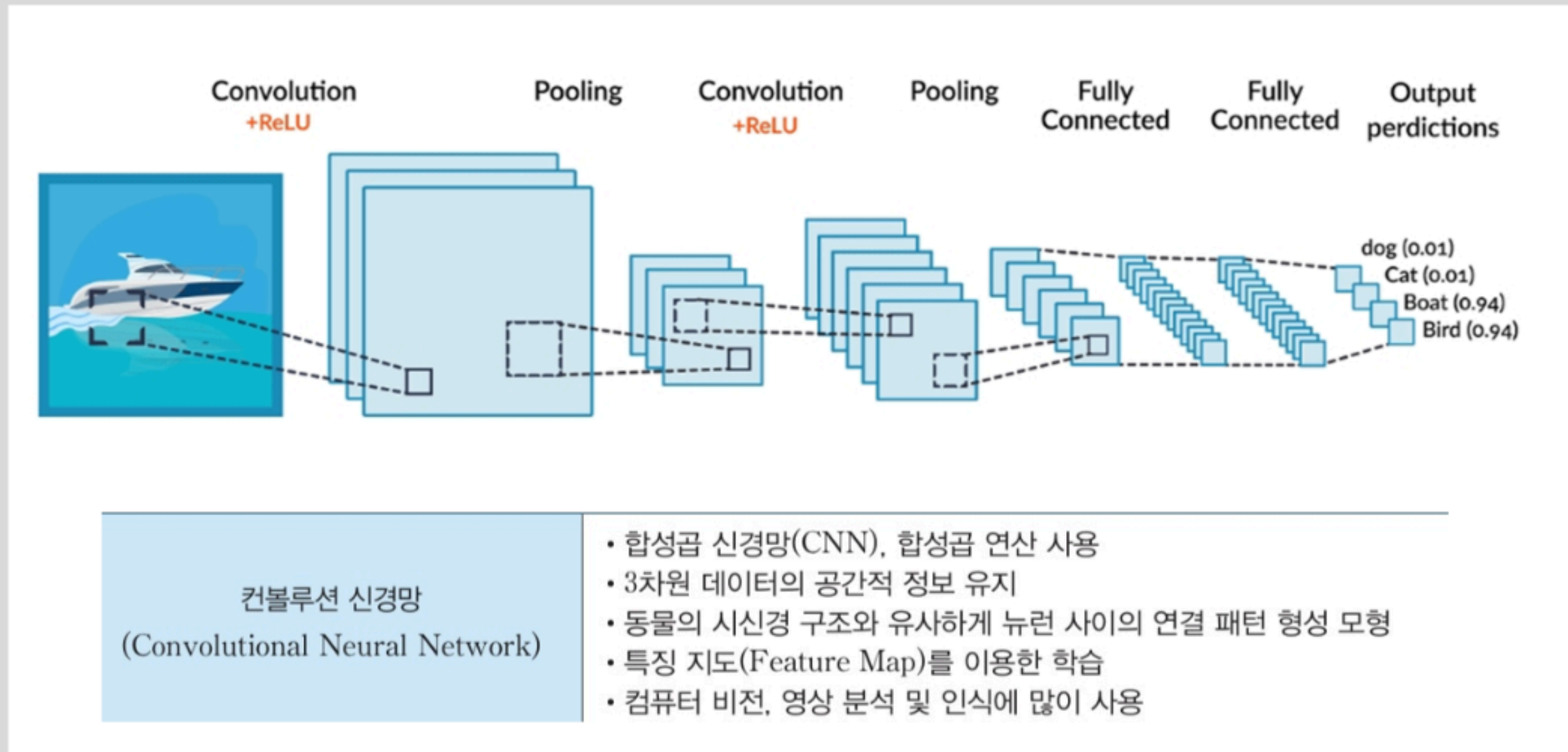
anger : 8000 images
anxiety : 8000 images
happy : 8000 images
naturally : 8592 images
panic : 8000 images
sad : 8000 images
wound : 8016 images

```
folder_path2 = '/content/drive/MyDrive/Colab Notebooks/ICTStudy/f
expressions = os.listdir(folder_path2)
expression_counts = []
for expression in expressions:
    expression_path = os.path.join(folder_path2, expression)
    num_images = len(os.listdir(expression_path))
    print(f"{expression} : {num_images} images")
    expression_counts.append(num_images)
```

anger : 2000 images
anxiety : 2000 images
happy : 2000 images
naturally : 2595 images
panic : 2000 images
sad : 2000 images
wound : 2022 images

CNN을 활용한 안면이미지 표정 감정 분석 모델

CNN모델 사용 이유



- 높은 정확도로 분류 작업 수행 가능
- 이미지의 로컬 패턴 추출하고 중요한 시각적 정보 강조
- 이미지 내의 중요한 구조와 패턴을 인식, 모델이 이미지나 비디오를 보다 효과적으로 분석할 수 있도록 도움

CNN을 활용한 안면이미지 표정 감정 분석 모델

Training Data 70,000건을 사용하여 모델 구현

```
no_of_classes = len(emotions)
picture_size = 128
batch_size = 128
epochs = 100
l2_reg = 0.0001 # L2 정규화 강도 설정

# 모델 생성 함수
def create_model(input_shape=(picture_size, picture_size, 1), no_of_classes=no_of_classes,
                 l2_reg=l2_reg):
    model = Sequential()

    # 1층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=input_shape,
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.3))

    # 2층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.3))

    # 3층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(128, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.4))

    # 4층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(256, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.4))
```

```
# CNN -> Flatten() -> DNN
model.add(Flatten())

# 5층 - Dense -> BatchNorm -> Activation -> Dropout
model.add(Dense(512, kernel_regularizer=l2(l2_reg)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))

# 6층 - Dense -> BatchNorm -> Activation -> Dropout
model.add(Dense(512, kernel_regularizer=l2(l2_reg)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))

# 출력층 - Dense -> Activation
model.add(Dense(no_of_classes, activation='softmax'))

# 모델 컴파일
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

return model
```

추가된 Dense층

- L2 정규화: 각 Conv2D와 Dense 층에서 `kernel_regularizer=l2(l2_reg)`를 사용하여 과적합을 방지
- 깊이와 복잡성 증가: 더 많은 필터(32, 64, 128, 256)와 더 큰 Dense 층(512)을 사용하여 네트워크의 표현력을 강화

CNN을 활용한 안면이미지 표정 감정 분석 모델

Training Data 70,000건을 사용하여 모델 구현

```
# 훈련용 이미지 데이터 생성 (흑백 이미지)
train_set = train_generator.flow_from_directory(
    directory=os.path.join(folder_path, 'train'),
    target_size=(picture_size, picture_size),
    color_mode='grayscale', # 흑백 이미지로 변환
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=True,
    seed=10
)

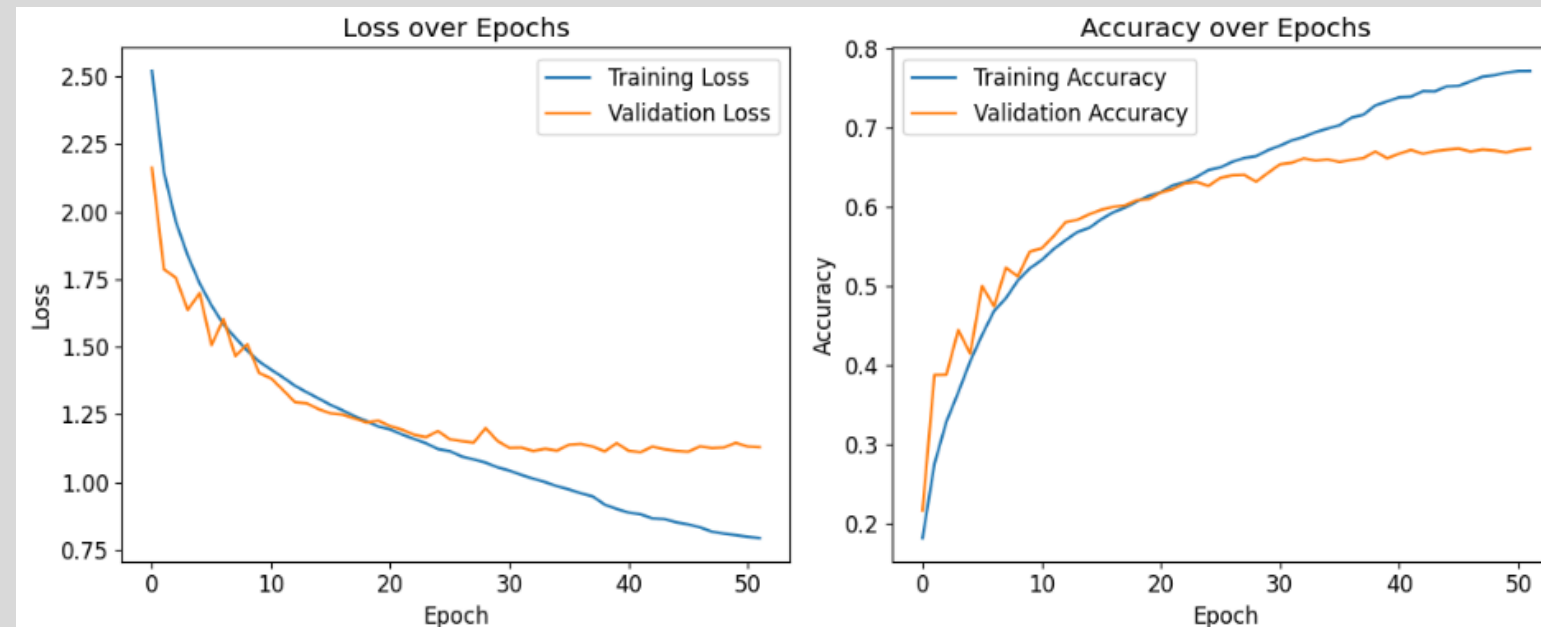
# 테스트용 이미지 데이터 생성 (흑백 이미지)
test_set = test_generator.flow_from_directory(
    directory=os.path.join(folder_path, 'test'),
    target_size=(picture_size, picture_size),
    color_mode='grayscale', # 흑백 이미지로 변환
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=True,
    seed=10
)
```

- color_mode='grayscale'을 사용해 이미지를 흑백으로 변환하여 입력

CNN을 활용한 안면이미지 표정 감정 분석 모델

Training Data 70,000건을 사용하여 모델 구현

Test Accuracy: **67.319%**



```
import matplotlib.pyplot as plt

# 학습 및 검증 손실 시각화
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss over Epochs')

# 학습 및 검증 정확도 시각화
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy over Epochs')

plt.tight_layout()
plt.show()
```

- 초기 테스트 정확도 62.31%에서 최종 테스트 정확도 67.32%로, 정확도는 약 5% 포인트 향상되었음

CNN을 활용한 안면이미지 표정 감정 분석 모델

모델 학습 시 파라미터 변경 후 성능 비교

Epochs: **20**
Batch Size: **64**
Learning Rate: 기본 Adam
Test Accuracy: **62.01%**

Epochs: **100**
Batch Size: **128**
Learning Rate: **0.0001**
Test Accuracy: **67.31%**

- 실패한 모델은 기본적인 CNN 구조로, 학습과정에서 모델 성능 제한적
- 성공한 모델은 더 깊고 복잡한 구조, L2 정규화 및 흑백 이미지 처리 등 다양한 최적화로 성능을 크게 개선했음

KoBert 를 활용한 문장 내 감정 인식

데이터 확인



#한국어 대화 음성 #감정 라벨링 #다분류 감정

감정 분류를 위한 대화 음성 데이터셋

분야 미분류 유형 오디오

조회수: 16,222 다운로드: 1,486 용량: 14.41 GB

데이터 통계

1차 공개 구축양

- 4차년도 14,606건
- 5차년도 10,011건
- 5차년도(2차) 19,374건

1.데이터 로드 및 컬럼명 확인

```
[14] import pandas as pd

path = '/content/merge_data.csv'
data = pd.read_csv(path, encoding='cp949')

[15] data.columns

Index(['wav_id', '발화문', '상황', '1번 감정', '1번 감정세기', '2번 감정', '2번 감정세기', '3번 감정', '3번 감정세기', '4번 감정', '4번 감정세기', '5번 감정', '5번 감정세기', '나이', '성별'],
      dtype='object')
```

2.감정 라벨 데이터 확인

```
[16] data_df = data[['발화문', '1번 감정', '2번 감정', '3번 감정', '4번 감정', '5번 감정']]
data_df.head(3)
```

	발화문	1번 감정	2번 감정	3번 감정	4번 감정	5번 감정
0	개를 예쁘다고 사놓고 끝까지 키우지도 않고 버리는 사람들이 엄청 많아졌다.	Angry	Angry	Angry	Angry	Angry
1	지금도 그대로 있어. 치우는 사람이 없어.	Neutral	Disgust	Sadness	Disgust	Disgust
2	맞아. 무기력증인 것 같아. 한동안 정말 바빴었거든.	Sadness	Sadness	Sadness	Sadness	Sadness

3.감정 라벨 개수 확인

```
data_df['1번 감정'].value_counts()
```

count

1번 감정

Sadness	3998
---------	------

Neutral	2498
---------	------

Disgust	1212
---------	------

Fear	1114
------	------

Angry	1012
-------	------

Happiness	121
-----------	-----

Surprise	56
----------	----

dtype: int64

4.감정 라벨 벡터화

```
# 감정 클래스 레이블을 숫자로 변환
```

```
emotion_mapping = {
    "Fear": 0,
    "Surprise": 1,
    "anger": 2,
    "Angry": 2,
    "sad": 3,
    "Sadness": 3,
    "Neutral": 4,
    "Happiness": 5,
    "Disgust": 6
}
```

```
data_df['1번 감정'] = data_df['1번 감정'].map(emotion_mapping)
data_df['1번 감정'].value_counts()
```

```
<ipython-input-22-ba4a5386ae29>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/
data_df['1번 감정'] = data_df['1번 감정'].map(emotion_mapping)
```

count

1번 감정

3	3998
---	------

4	2498
---	------

6	1212
---	------

0	1114
---	------

2	1012
---	------

5	121
---	-----

1	56
---	----

dtype: int64

KoBert 를 활용한 문장 내 감정 인식

데이터 라벨 전처리

대, 소문자가 혼용되어 있어 모든 라벨을 소문자로 전처리

```
df1 = pd.read_csv('/content/5차년도.csv', encoding='cp949')
df2 = pd.read_csv('/content/4차년도.csv', encoding='cp949')
df3 = pd.read_csv('/content/5차년도_2차.csv', encoding='cp949')
```

```
print(df1['1번 감정'].value_counts())
print(df2['1번 감정'].value_counts())
print(df3['1번 감정'].value_counts())
```

```
1번 감정
Sadness      3998
Neutral      2498
Disgust       1212
Fear         1114
Angry         1012
Happiness      121
Surprise       56
Name: count, dtype: int64
```

```
1번 감정
Sadness      6593
Angry        3374
Neutral      2635
Disgust       913
Fear          645
Happiness     322
Surprise     124
Name: count, dtype: int64
```

```
1번 감정
neutral      6825
happiness    3066
sadness      2824
disgust      2252
surprise     1739
fear         1351
angry        1317
Name: count, dtype: int64
```

데이터 결합

```
data_merge = pd.concat([df1, df2, df3])
```

'1번 감정' 컬럼의 값을 소문자로 변환

```
data_merge['1번 감정'] = data_merge['1번 감정'].str.lower()
```

변환된 결과를 확인하기 위해 value_counts()를 다시 실행

```
print(data_merge['1번 감정'].value_counts())
```

```
1번 감정
sadness      13415
neutral      11958
angry         5703
disgust       4377
happiness     3509
fear          3110
surprise      1919
Name: count, dtype: int64
```

```
data_merge['1번 감정'].value_counts()
```

count

1번 감정

sadness	13415
neutral	11958
angry	5703
disgust	4377
happiness	3509
fear	3110
surprise	1919

dtype: int64

KoBert 를 활용한 문장 내 감정 인식

데이터 증강 (KoBert를 사용하여 단어 위치를 조정하여 다양화)

```
# 필요한 모듈 import
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from tqdm.notebook import tqdm

from transformers import AdamW, BertTokenizer, BertModel, get_cosine_schedule_with_warmup

# KoBERT 모델과 토큰라이저를 불러옵니다.
tokenizer = BertTokenizer.from_pretrained('monologg/kobert')
bertmodel = BertModel.from_pretrained('monologg/kobert')
print("KoBERT 모델과 토큰라이저 로드 완료")
```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
tokenizer_config.json: 100% ██████████ 51.0/51.0 [00:00<00:00, 3.91kB/s]
vocab.txt: 100% ██████████ 77.8k/77.8k [00:00<00:00, 1.04MB/s]
config.json: 100% ██████████ 426/426 [00:00<00:00, 35.5kB/s]
model.safetensors: 100% ██████████ 369M/369M [00:01<00:00, 347MB/s]
KoBERT 모델과 토큰라이저 로드 완료

참고 : <https://bbarry-lee.github.io/ai-tech/KoBERT%EB%A5%BC-%ED%99%9C%EC%9A%A9%ED%95%9C-%EA%B0%90%EC%A0%95%EB%B6%84%EB%A5%98-%EB%AA%A8%EB%8D%B8-%EA%B5%AC%ED%98%84.html>

라벨 값 비율이 고르지 못한 것을 확인 후 텍스트 데이터 증강

- 값이 가장 많은 sad 기준으로 증강 시도
- 런타임이 끊겨서 제대로 진행되지 않음

KoBert 를 활용한 문장 내 감정 인식

데이터 증강 (KoBert를 사용하여 단어 위치를 조정하여 다양화)

```
if __name__ == "__main__":  
    # 데이터 불러오기  
    df = pd.read_csv(f'{path}/emotion_vec.csv')  
  
    # 각 라벨의 타겟 개수 설정  
    target_count = 14000  
    iter_count = 2 # 각 텍스트에 대해 증강 작업을 몇 번 수행할지 설정  
  
    # 각 라벨에 대해 증강 수행  
    unique_labels = df['label'].unique()  
    for label in unique_labels:  
        df = textAug(df, label, target_count, iter_count)  
  
    filename = f'{path}/textaug_completed_dataframe_0808.csv'  
    df.to_csv(filename, index=False)  
    print(f'Data augmentation completed. Saved to {filename}')
```

```
Augmentations: 0%|          | 0/2 [00:00<?, ?it/s]  
Augmentations: 50%|██████    | 1/2 [00:00<00:00, 6.35it/s]  
Augmentations: 100%|██████████| 2/2 [00:00<00:00, 6.70it/s]  
Rows Processing: 46%|██████   | 2087/4548 [1:34:38<29:53, 1.37it/s]  
Augmentations: 0%|          | 0/2 [00:00<?, ?it/s]  
Augmentations: 50%|██████    | 1/2 [00:02<00:02, 2.89s/it]  
Augmentations: 100%|██████████| 2/2 [00:04<00:00, 2.20s/it]  
Rows Processing: 46%|██████   | 2088/4548 [1:34:43<1:17:42, 1.90s/it]
```

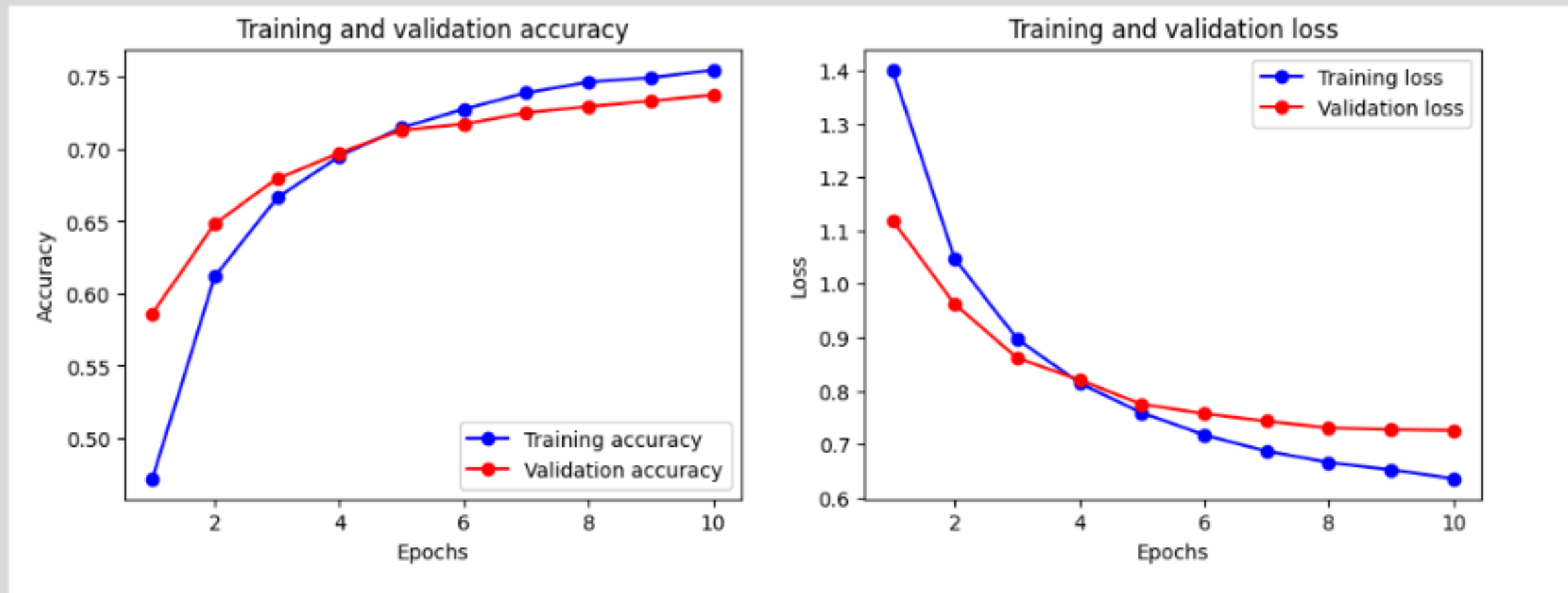
```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ICTStudy/ATeam/incredata/emotionData_final.csv')  
df['label'].value_counts()
```

count	
label	
0	16000
3	16000
2	16000
5	16000
6	16000
4	16000
1	16000

- 3000 ~ 4000 개 사이 값을 가지는 라벨이 많아 4000개 기준으로 증강 후 10000개로 다시 증강 시도함
- 런타임 유지를 위해 100개씩 40번 반복, 100개씩 100번 반복함

KoBert 를 활용한 문장 내 감정 인식

데이터 증강한 데이터 셋으로 재학습 (과대적합으로 확인되어 잘못된 학습 확인)

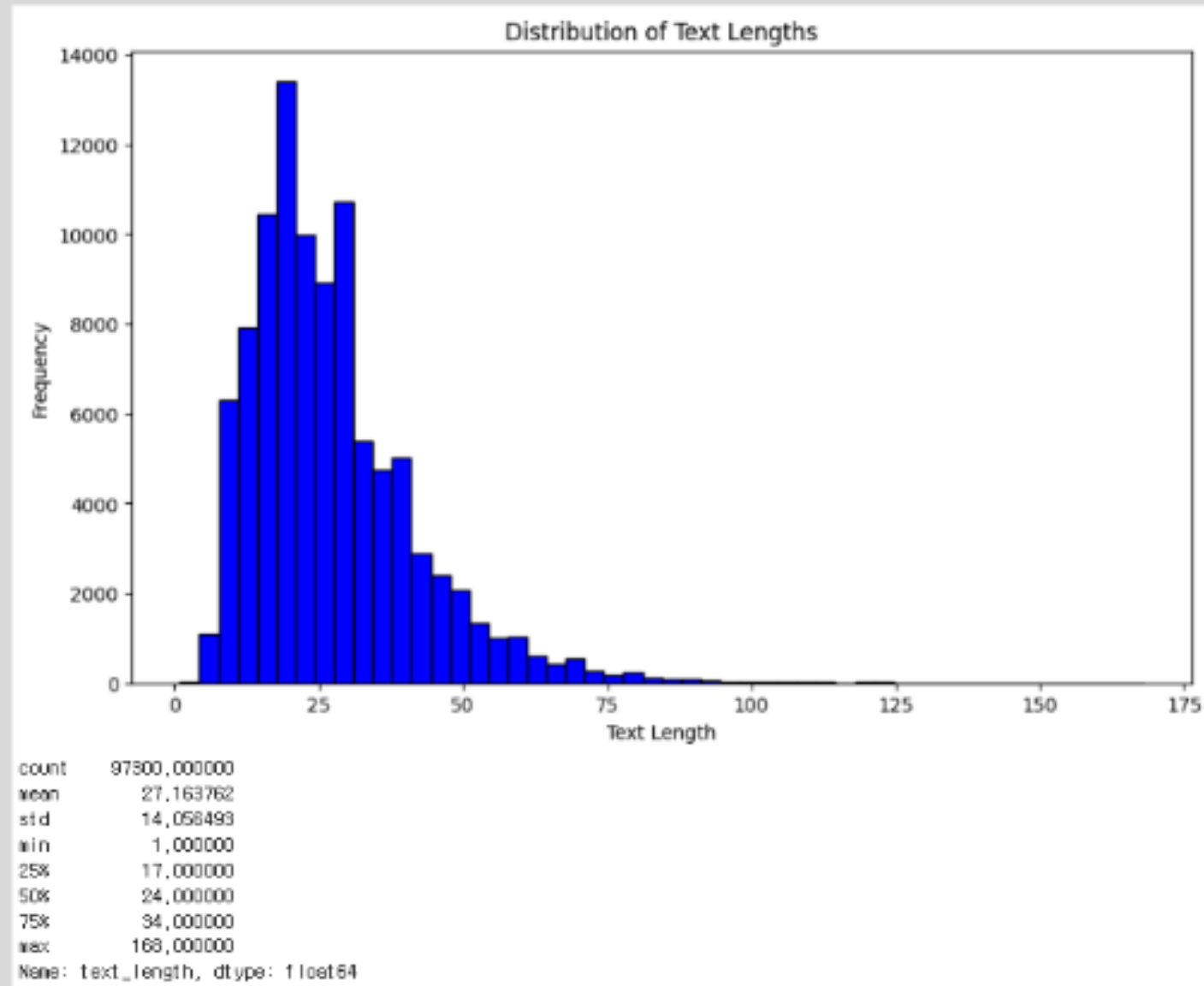


```
learning(max_len=64,  
batch_size=32,  
warmup_ratio=0.03,  
num_epochs=10,  
max_grad_norm=1,  
log_interval=200,  
learning_rate=3e-5)
```

- 동일한 파라미터로 학습
- 여러번 테스트 하기 위해 학습, 평가, 시각화 코드를 함수화함

KoBert 를 활용한 문장 내 감정 인식

데이터 정보 확인 (최대 길이 수치 확인)



```
# 데이터 로드
data = pd.read_csv(f'{path}/emotionData_final.csv')

# 텍스트 길이 계산
data['text_length'] = data['text'].apply(len)

# 텍스트 길이 분포 시각화
plt.figure(figsize=(10, 6))
plt.hist(data['text_length'], bins=50, color='blue', edgecolor='black')
plt.title('Distribution of Text Lengths')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.show()

# 텍스트 길이의 통계값 확인
length_stats = data['text_length'].describe()
print(length_stats)
```

- 최소, 최대 텍스트 길이를 확인하고 조정해야 할 지 판단
- 25% ~ 75% 사이 길이를 가진 데이터를 추출하여 사용

KoBert 를 활용한 문장 내 감정 인식

데이터 전처리 (최소, 최대 텍스트에 대해 임의 값을 추가하거나 자름)

	text
0	일단은 취소를 한번 해보고, 그게 안되면 약속 장소를 명동이 아닌 좀 가깝나 이런...
1	나 엘리베이터에 갇혔어.
2	식당에서 친구랑 밥먹고 있었거든. 근데 막 지진이 일어났어.
3	어디가 좋을까?
4	아, 너무 너무 무서워. 나 어쩌면 좋지? 119에 전화해봐야겠어.

	cleaned_text	adjusted_text
0	일단은 취소를 한번 해보고 그게 안되면 약속 장소를 명동이 아닌 좀 가깝나 이런 곳...	일단은 취소를 한번 해보고 그게 안되면 약속 장소를 명동이 아닌 좀 가까...
1	나 엘리베이터에 갇혔어	나 엘리베이터에 갇혔어 [짧음]
2	식당에서 친구랑 밥먹고 있었거든 근데 막 지진이 일어났어	식당에서 친구랑 밥먹고 있었거든 근데 막 지진이 일어났어
3	어디가 좋을까	어디가 좋을까 [짧음]
4	아 너무 너무 무서워 나 어쩌면 좋지 에 전화해봐야겠어	아 너무 너무 무서워 나 어쩌면 좋지 에 전화해봐야겠어

```
learning(max_len=64,
        batch_size=32,
        warmup_ratio=0.03,
        num_epochs=10,
        max_grad_norm=1,
        log_interval=200,
        learning_rate=3e-5)

# 데이터 로드
data = pd.read_csv(f'{path}/emotionData_final.csv')

# 텍스트 정제 함수
def clean_text(text):
    text = re.sub(r'^가~할\s', '', text) # 특수 문자 제거
    text = re.sub(r'\s+', ' ', text) # 공백 정리
    return text.strip()

# 텍스트 길이 조정 함수
# def adjust_text_length(text, min_length=17, max_length=34): 25% ~ 75% 로 범위 조절
def adjust_text_length(text, min_length=17, max_length=34):
    length = len(text)

    if length < min_length:
        # 짧은 텍스트에 대해 특수 토큰을 추가하거나 보완
        return text + ' [짧음]'
    elif length > max_length:
        # 긴 텍스트는 자르거나 요약
        return text[:max_length] + '...'
    else:
        return text

# 데이터에 전처리 적용
data['cleaned_text'] = data['text'].apply(clean_text)
data['adjusted_text'] = data['cleaned_text'].apply(adjust_text_length)

# 원본 label과 함께 새로운 CSV 파일로 저장
output_file_path = 'adjusted_data_with_labels_0810.csv'
data[['text', 'cleaned_text', 'adjusted_text', 'label']].to_csv(output_file_path, index=False)

# 결과 확인
data[['text', 'cleaned_text', 'adjusted_text', 'label']].head()
```

KoBert 를 활용한 문장 내 감정 인식

데이터 전처리 (길이를 조정한 데이터의 성능 비교)

Original Data - Accuracy: 0.9304, F1 Score: 0.9304, Precision: 0.9307, Recall: 0.9304
Adjusted Data - Accuracy: 0.9254, F1 Score: 0.9255, Precision: 0.9262, Recall: 0.9254

```
# 모델 학습 및 예측 (원본 데이터)
model_orig = MultinomialNB()
model_orig.fit(X_train_orig_vec, y_train_orig)
y_pred_orig = model_orig.predict(X_test_orig_vec)

# 모델 학습 및 예측 (조정된 데이터)
model_adj = MultinomialNB()
model_adj.fit(X_train_adj_vec, y_train_adj)
y_pred_adj = model_adj.predict(X_test_adj_vec)

# 성능 평가 (원본 데이터)
accuracy_orig = accuracy_score(y_test_orig, y_pred_orig)
f1_orig = f1_score(y_test_orig, y_pred_orig, average='weighted')
precision_orig = precision_score(y_test_orig, y_pred_orig, average='weighted')
recall_orig = recall_score(y_test_orig, y_pred_orig, average='weighted')

# 성능 평가 (조정된 데이터)
accuracy_adj = accuracy_score(y_test_adj, y_pred_adj)
f1_adj = f1_score(y_test_adj, y_pred_adj, average='weighted')
precision_adj = precision_score(y_test_adj, y_pred_adj, average='weighted')
recall_adj = recall_score(y_test_adj, y_pred_adj, average='weighted')

# 결과 비교 출력
print("Original Data - Accuracy: {:.4f}, F1 Score: {:.4f}, Precision: {:.4f}, Recall: {:.4f}".format(accuracy_orig, f1_orig, precision_orig, recall_orig))
print("Adjusted Data - Accuracy: {:.4f}, F1 Score: {:.4f}, Precision: {:.4f}, Recall: {:.4f}".format(accuracy_adj, f1_adj, precision_adj, recall_adj))
```

- 데이터 재확인 : 텍스트 길이 조정이 모델 성능에 미치는 영향 비교
- 원본 수치가 더 좋아서 원본 데이터를 사용하기로 함

KoBert 를 활용한 문장 내 감정 인식

Max Length 중앙 값으로 구성하여 학습 진행

```
Model saved at epoch 9 with test accuracy: 0.7300347222222222
Training Epoch 10: 0%|          | 1/609 [00:00<08:15, 1.23it/s]epoch 10 batch id 1 loss 0.7101476192474365 train acc 0.71875
Training Epoch 10: 33%|███      | 201/609 [01:48<03:39, 1.86it/s]epoch 10 batch id 201 loss 0.7512655854225159 train acc 0.7475124378109452
Training Epoch 10: 66%|██████    | 401/609 [03:36<01:52, 1.84it/s]epoch 10 batch id 401 loss 0.7042497992515564 train acc 0.7448176433915212
Training Epoch 10: 99%|██████████| 601/609 [05:24<00:04, 1.86it/s]epoch 10 batch id 601 loss 0.6305090188980103 train acc 0.7455282861896838
Training Epoch 10: 100%|██████████| 609/609 [05:28<00:00, 1.86it/s]
epoch 10 train acc 0.7456896551724138 train loss 0.657299041062936
Evaluating Epoch 10: 100%|██████████| 153/153 [00:27<00:00, 5.60it/s]
epoch 10 test acc 0.7340175653594772 test loss 0.7191685075853386
```

GPU 메모리 사용량: 1.46 GB

Model saved at epoch 10 with test accuracy: 0.7340175653594772

학습 파라미터 설정 예시

```
max_len = 128 # 또는 256, 512 등 실험을 통해 최적화
batch_size = 128 # GPU 메모리 상황에 따라 128, 256, 512
learning_rate = 2e-5 # 또는 3e-5, 5e-5 등 실험적으로 최적화
num_epochs = 10 # 충분한 학습을 위해 설정, 조기 종료에 있을 경우 적당한 값 설정
max_grad_norm = 1.0 # Gradient clipping을 위한 최대 값 설정
log_interval = 200 # 학습 로그를 출력할 간격 설정
warmup_ratio = 0.1 # 학습 초기 단계에서의 warmup 비율 설정
learning(max_len, batch_size, warmup_ratio, num_epochs, max_grad_norm, log_interval, learning_rate)
```

```
import torch
```

```
# 가상으로 모델과 데이터 로드
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = BERTClassifier(bertmodel, dr_rate=0.5).to(device)
```

```
dataset_train, dataset_test = train_test_split(data_list, test_size=0.2, shuffle=True,
random_state=32)
```

```
data_train = BERTDataset(dataset_train, 0, 1, tokenizer, max_len, True, False)
```

```
# 메모리 체크를 위해 임의의 배치 크기 설정
```

```
batch_size = 64 # 초기 설정, 이후 조정 가능
```

```
train_dataloader = DataLoader(data_train, batch_size=batch_size, num_workers=5,
shuffle=True)
```

```
# GPU 메모리 점검
```

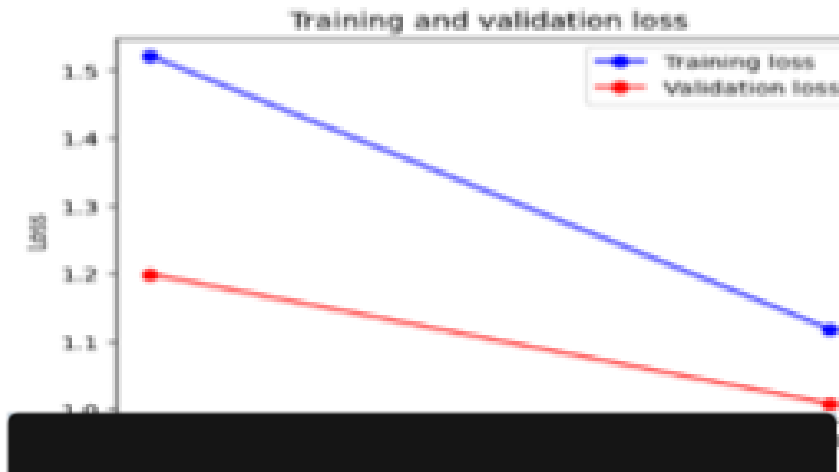
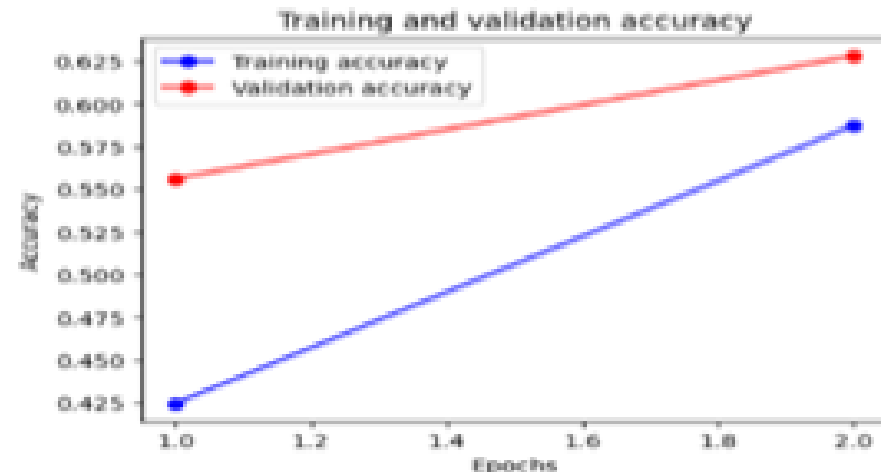
```
print(f"GPU 메모리 사용량: {torch.cuda.memory_allocated() / 1024 ** 3:.2f} GB")
```

KoBert 를 활용한 문장 내 감정 인식

Max Length 중앙 값으로 구성하여 학습 진행 (GPU 메모리 사용량 확인)

```
Training Epoch 2: 100% ██████████ 609/609 [05:28<00:00, 1.85k/s]
epoch 2 batch id 1 loss 1.2016433477401733 train acc 0.5390625
epoch 2 batch id 201 loss 1.1662259101067676 train acc 0.5621119164079602
epoch 2 batch id 401 loss 1.0336960554122925 train acc 0.575475374064636
epoch 2 batch id 601 loss 0.9977717399597168 train acc 0.5665464606985025
epoch 2 train acc 0.5673614532019704 train loss 1.116659624040536
Evaluating Epoch 2: 100% ██████████ 113/113 [00:27<00:00, 3.66k/s]
epoch 2 test acc 0.6261147675816994 test loss 1.0065619765946626

GPU 메모리 사용량: 5.21 GB
조기 종료
Model saved at epoch 2 with test accuracy: 0.6261147675816994
```



```
max_len = 128
batch_size = 128
learning_rate = 3e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

```
model_name = 'test_kobert_early2'
max_len = 128
batch_size = 128
learning_rate = 3e-5
for i in range(len(list_sentence)):
    print(predict(list_sentence[i], model_name, max_len, batch_size))
```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>).
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100% ██████████ 51.0/51.0 [00:00<00:00, 3.60kB/s]
vocab.txt: 100% ██████████ 77.8k/77.8k [00:00<00:00, 3.56MB/s]
config.json: 100% ██████████ 426/426 [00:00<00:00, 30.7kB/s]
model.safetensors: 100% ██████████ 369M/369M [00:01<00:00, 266MB/s]

```
내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 여기 조금 무성다 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 진짜 예쁘다 >> 입력하신 내용에서 공포가 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 슬픔이 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 슬픔이 느껴집니다.
None
내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 나 최우수상 탔어! >> 입력하신 내용에서 행복이 느껴집니다.
None
내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 놀람이 느껴집니다.
None
```

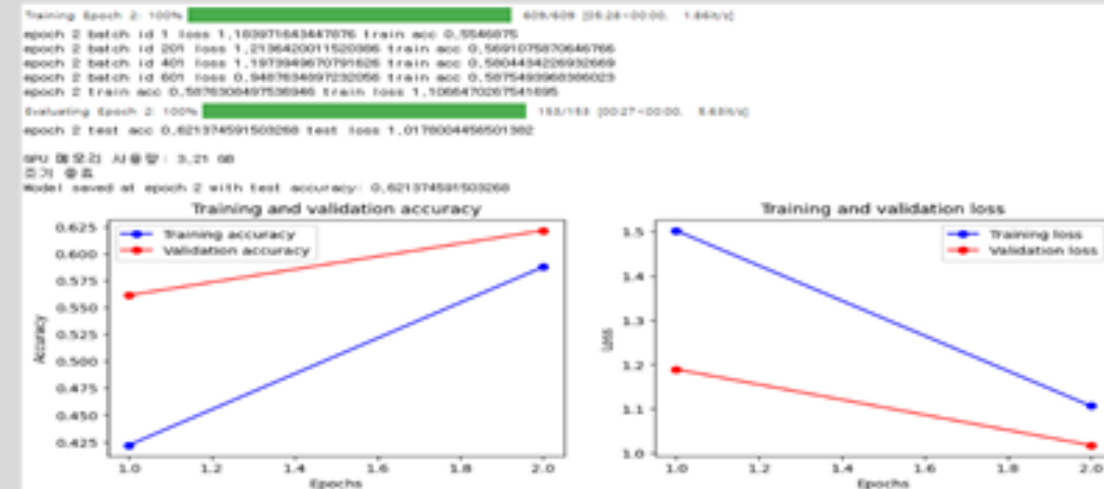
KoBert 를 활용한 문장 내 감정 인식

파라미터 테스트 (Max Length, Batch Size, Learning Rate 변경하여 확인)

```
max_len = 128
batch_size = 128
learning_rate = 5e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

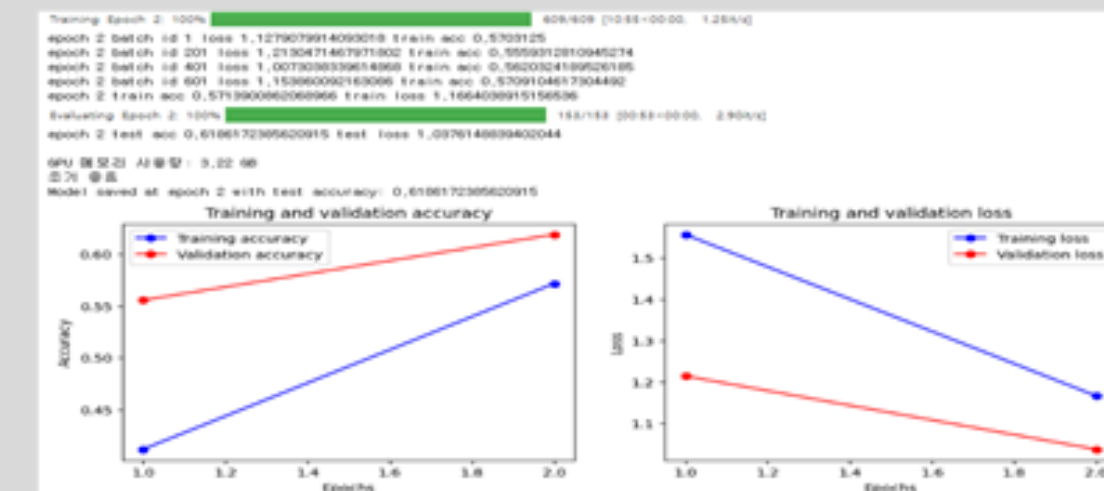
```
max_len = 256
batch_size = 128
learning_rate = 2e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

```
max_len = 256
batch_size = 128
learning_rate = 2e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```



```
model_name = 'test_kobert_early3'
max_len = 128
batch_size = 128
learning_rate = 5e-5
for i in range(len(list_sentence)):
    print( predict(list_sentence[i], model_name, max_len, batch_size))
```

내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 여기 조금 무성다 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 잔잔 예쁘다 >> 입력하신 내용에서 혐오가 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 공포가 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 행복이 느껴집니다.
None
내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
None
내용 : 나 최우수상 받아! >> 입력하신 내용에서 공포가 느껴집니다.
None
내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 놀람이 느껴집니다.
None

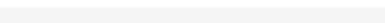


```
model_name = 'test_kobert_early4'
max_len = 256
batch_size = 128
learning_rate = 2e-5
for i in range(len(list_sentence)):
    print( predict(list_sentence[i], model_name, max_len, batch_size))
```

내용 : 우와 무지개다 대박 >> 입력하신 내용에서 혐오가 느껴집니다.
None
내용 : 여기 조금 무성다 >> 입력하신 내용에서 혐오가 느껴집니다.
None
내용 : 잔잔 예쁘다 >> 입력하신 내용에서 공포가 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 공포가 느껴집니다.
None
내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 행복이 느껴집니다.
None
내용 : 우와 무지개다 대박 >> 입력하신 내용에서 혐오가 느껴집니다.
None
내용 : 나 최우수상 받아! >> 입력하신 내용에서 행복이 느껴집니다.
None
내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 혐오가 느껴집니다.
None

KoBERT 모델과 토크나이저 로드 완료

/usr/local/lib/python3.10/dist-packages/transformers/optimization.py:591: FutureWarning: This implementation of AdamW is deprecated. Please use the one in transformers.optimization.py

Training Epoch 1: 0%  0/609 [00:01<?, ?it/s]

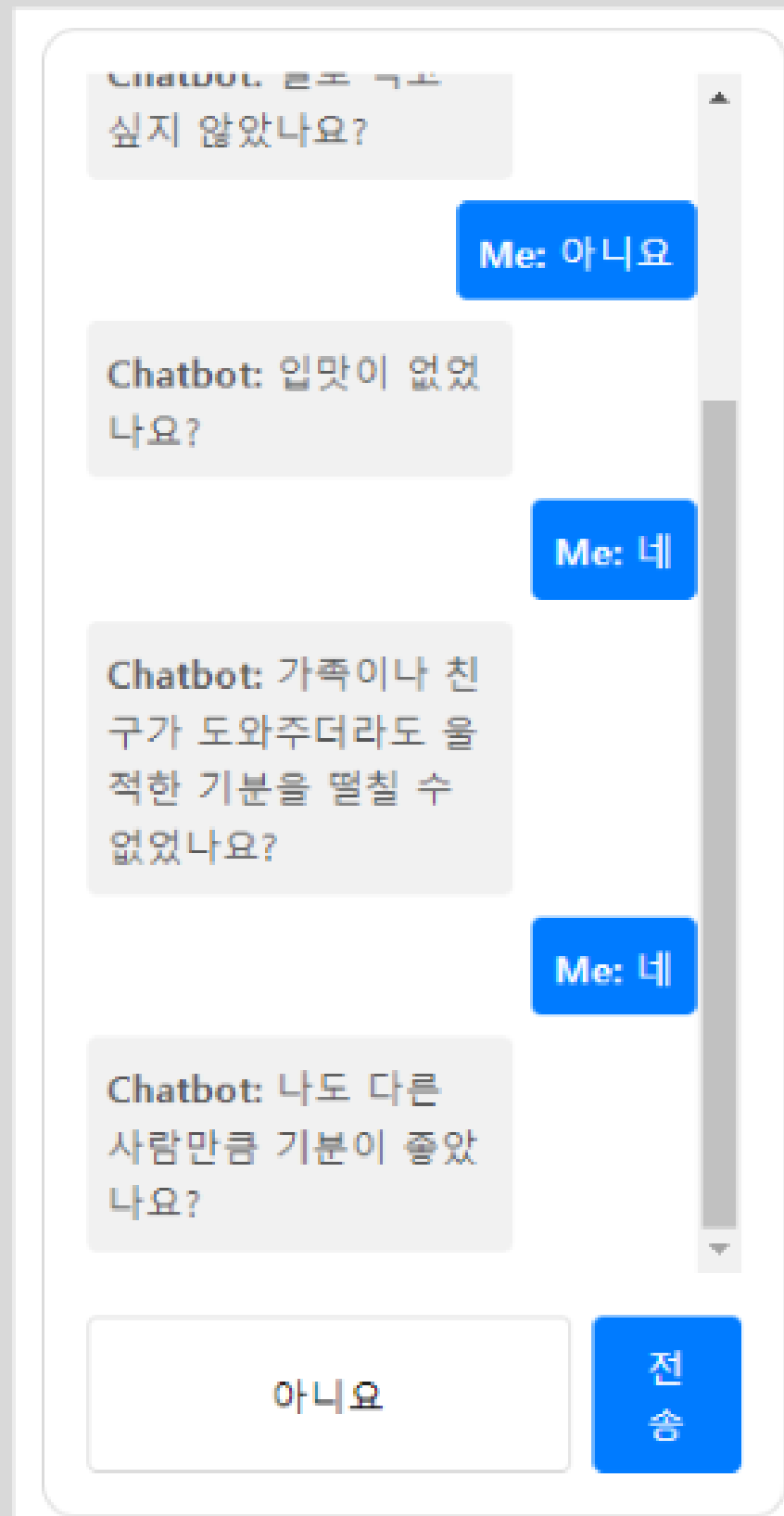
```
OutOfMemoryError                                Traceback (most recent call last)
<ipython-input-56-585b0fed6735> in <cell line: 8>()
      6 log_interval = 200 # 학습 로그를 출력할 간격 설정
      7 warmup_ratio = 0.1 # 학습 초기 단계에서의 warmup 비율 설정
----> 8 learning(max_len, batch_size, warmup_ratio, num_epochs, max_grad_norm, log_interval, learning_rate, try_count)
      9
     10 try_count += 1
```

```
/usr/local/lib/python3.10/dist-packages/transformers/activations.py in forward(self, input)
      76
      77 def forward(self, input: Tensor) -> Tensor:
----> 78     return self.act(input)
      79
      80
```

OutOfMemoryError: CUDA out of memory. Tried to allocate 768.00 MiB. GPU

CES-D 설문을 활용한 심리진단 서비스

채팅형 심리진단 서비스 (Controller, Vue (TTS))



```
@RestController
@RequestMapping("/survey")
public class SimpletestController {
    @GetMapping("/getRespTotal")
    public int getRespTotal(@RequestParam("userid") String userid,
                           @RequestParam("respdate") String respdate) {
        int score = simpletestService.getRespTotal(userid, respdate);
        int state = 0;
        if (10 <= score) {
            state = 1;
        } else if (5 <= score) {
            state = 2;
        } else if (0 <= score) {
            state = 3;
        } else {
            state = 0;
        }
        return state;
    }
}

@GetMapping("/responses")
public List<Map<String, Object>> getUserResponses(@RequestParam("userid") String userId) {
    return simpletestService.getUserResponses(userId);
}

async fetchSurveyQuestions() {
    try {
        const response = await
        axios.get(`${process.env.VUE_APP_BACK_END_URL}/survey/getQuestion`);
        this.surveyQuestions = response.data;
        this.addChatbotMessage(this.surveyQuestions[this.currentQuestionIndex].question);
    }

    // 텍스트를 음성으로 변환해주는 함수
    speak(text) {
        const msg = new SpeechSynthesisUtterance(text);
        msg.lang = 'ko-KR'; // 한국어 설정
        // 음성을 재생
        window.speechSynthesis.speak(msg);
    },
}
```

CES-D 설문을 활용한 심리진단 서비스

사용자의 응답을 정수형 점수로 환산 (Vue)



```
// 사용자 응답을 서버에 저장하는 함수
async sendResponses() {
  try {
    const userid = 'user2';
    const responseDate = new Date().toISOString().split('T')[0];
    for (const response of this.responses) {
      await axios.post(`${process.env.VUE_APP_BACKEND_URL}/survey/insertResponse`, {
        userid: userid,
        surveycode: this.surveyCode,
        qnum: response.qnum,
        respscore: response.respscore,
        respdate: responseDate
      });
    }
  } catch (error) {
    console.error('Error saving responses:', error);
  }
}
```



```
// 사용자가 입력한 메시지의 점수를 계산하는 함수
calculateScore(text) {
  const positiveWords = ['<', '네', '응', '매우', '엄청', '맞아'];
  const lowerText = text.toLowerCase();
  return positiveWords.some(word => lowerText.includes(word)) ? 1 : 0;
},

// 메시지 창을 자동으로 스크롤하는 함수
scrollToEnd() {
  const container = this.$el.querySelector('.messages');
  container.scrollTop = container.scrollHeight;
},

startRecording() {
  navigator.mediaDevices.getUserMedia({ audio: true })
    .then(stream => {
      this.mediaRecorder = new MediaRecorder(stream);
      this.mediaRecorder.ondataavailable = event => {
        this.audioChunks.push(event.data);
      };
      this.mediaRecorder.onstop = this.handleStopRecording;
      this.mediaRecorder.start();
      this.isRecording = true;
    })
    .catch(error => {
      console.error('Error accessing media devices:', error);
      // 사용자에게 오류 메시지 표시 또는 다른 대응 방안 구현
    });
},
```

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

문의 등록 화면

문의하기

문의 등록시 유의사항

(☹_☹_☹) 클린봇이 상업적인 사진, 욕설, 음란성 문구를 AI 기술로 감지하고 있습니다.

test1

2024-08-27

카테고리를 선택해주세요

제목을 입력해주세요.

첨부파일

파일 선택 | 선택된 파일 없음

등록하기

목록으로

```
add() {  
  const formData = new FormData();  
  formData.append('qna', JSON.stringify(this.qna));  
  if (this.image) {  
    formData.append('file', this.image);  
  }  
  
  axios.post(`${process.env.VUE_APP_BACKEND_URL}/qna/qnaadd`, formData, {  
    headers: {  
      'Content-Type': 'multipart/form-data'  
    }  
  })  
  .then(response => {  
    this.qna = response.data;  
    this.$router.push(`/Qnalist`);  
  })  
}
```

- this.qna 객체를 JSON문자열로 변환한 후, formData에 file이라는 이름으로 이미지를 추가
- .then 콜백함수의 response는 정상적으로 this.qna에 담아냄

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

문의 등록 시 이미지 처리

첨부파일

파일 선택 상담사.png

안녕하세요 오늘은 중요한 수업이 있습니다.

등록하기 목록으로

 1fee58d5-91d7-4783-affe-8b5fc8fd8dc6.png	 62a62d0a-e528-4fc4-8879-184a4878000f.png	 94663ae5-bb73-4f35-b6bb-0305b9ae765a.png	 a4a56d60-22e7-4582-a0af-dc65b3512fc2.png	 a085f84e-c3a1-4b76-9d51-d14d042d3239.png	 aeb525bd-bd40-438e-bc7d-0d5009684356.jpg
 bfc01158-e3fe-43c7-91b3-94fa411cbf17.jpg	 c15743be-a856-4ff4-821f-9e38de589421.png	 f7950c2e-f7c0-411c-8f82-b55a710dbf94.png	 fbb94bac-4cd6-43cb-af06-1ce00e8a1bf8.jpg		

```
onFileChange(e) {  
  const formData = new FormData();  
  this.file = e.target.files[0];  
  formData.append("file1", this.file);  
  axios.post(`http://192.168.0.92:9000/thisisspam/insert_img`, formData, {  
    headers: {  
      'Content-Type': 'multipart/form-data'  
    }  
  })  
  .then((response) => {  
    this.image = this.file;  
    if (this.file) {  
      this.imagePreview = URL.createObjectURL(this.file);  
    }  
    console.log(response.data);  
    this.imgdata = JSON.stringify(response.data, null, 2);  
    this.imgname = e.target.files[0];  
  })  
  .catch((error) => {  
    console.error(error);  
    alert('이미지 처리 중 오류가 발생했습니다.');
```

- 이미지를 선택하고 이미지 데이터를 저장
- 이미지 인식 실패하면 에러 alert를 표시

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

Google Vision을 사용한 이미지 내 스팸단어 검출 서비스



```
file = request.FILES['file']
content = file.read()
image = vision.Image(content=content)
```



client 모델을 사용해 이미지에서 텍스트가 있는 영역 감지

```
response = client.text_detection(image=image)
texts = response.text_annotations
filtered_texts = []
for text in texts:
    cleaned_text = re.sub('[^가-힣-ㅇㅈㅊ-ㅣ|0-9]', '', text.description)
    cleaned_text = cleaned_text.replace(' ', '') # 모든 공백 제거
    if cleaned_text: # 빈 문자열이 아닌 경우만 추가
        filtered_texts.append(cleaned_text)
return filtered_texts
```

response.text_annotations는 감지된 텍스트와 주석을 포함해 디지털 형식으로 변환

```
re.sub('[^가- ㅈ-하-ㅣ|a-zA-Z0-9]', '', text)
```

-정규표현식을 사용해서 한글(완성형및자모),영어대소문자,숫자를 제외한 모든 문자를 공백으로 치환

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

Google Vision을 사용한 이미지 내 스팸단어 검출 서비스



```
def __init__(self):
    self.client = vision.ImageAnnotatorClient()
    self.vectorizer = CountVectorizer()
    self.model = GaussianNB()

def Project2Model(self, texts, labels):
    """텍스트와 레이블을 사용하여 모델을 학습합니다."""
    X = self.vectorizer.fit_transform(texts).toarray()
    y = labels
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
    self.model.fit(X_train, y_train)
    y_pred = self.model.predict(X_test)

myModel = Project2Model()
try:
    texts = myModel.detect_texts(file_path)
    print(f"Detected texts => {texts}")
    return JsonResponse(resJson)
except Exception as e:
    return JsonResponse({"error": f"텍스트 감지 중 오류 발생: {str(e)}"}, status=400)
if len(texts) < 3:
    return JsonResponse({"error": "텍스트 감지 실패"}, status=400)
```

- 가우시안나이프베이즈 초기화 및 모델 훈련 및 학습
- 모델 사용해서 텍스트 감지 Json Response 타입으로 반환
- file_path를 매개변수로 한 이미지에서 텍스트를 추출하여 반환

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

나의 문의 목록 보기

나의 문의

문의순서	카테고리	제목	작성날짜	답변여부	관리
1	회원관련	회원 관련	2024- 08-27	답변대기 중	<button>삭제</button>
2	심리진단	심리 진단	2024- 08-27	답변대기 중	<button>삭제</button>
3	서비스문의	서비스문의	2024- 08-27	답변대기 중	<button>삭제</button>

```
created() {
  this.fetchqna();
},
methods: {
  fetchqna() {
    axios.get(`${process.env.VUE_APP_BACK_END_URL}/qna/qnalist/${this.membernum}`)
      .then(response => {
        this.formDats = response.data;
      });
  },
  gdelete(num) {
    if (confirm("정말 삭제하시겠습니까?")) {
      axios.delete(`${process.env.VUE_APP_BACK_END_URL}/qna/qnadelete/${num}`)
        .then(() => {
          this.fetchqna();
        });
    }
  }
}
```

- fetchqna 함수가 실행되면 데이터를 가져오는 시도를 한다.
- gdelete 함수가 실행되면 알람을 띄움
- created()는 컴포넌트가 생성되자마자 호출을 해 데이터를 가져옴

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

나의 문의 상세 보기

문의 상세보기

문의 등록시 유의사항

상업적인 사진, 욕설, 음란성 글을 등록할 수 없습니다.

wjdgksmf

2024-08-27

회원관련

회원관련

회원관련

안녕하세요 오늘은 중요한 수업이 있습니다.

수정하기



```
created() {
  this.fetchQna();
},
fetchQna() {
  const num = this.$route.params.num;
  axios.get(`${process.env.VUE_APP_BACK_END_URL}/qna/qnadetail/${num}`)
    .then(response => {
      console.log("API Response:", response.data); // API 응답 확인
      this.formdata = response.data;
      if (this.formdata.imgname && this.formdata.imgname !== 'default.png') {
        this.image = `${process.env.VUE_APP_BACK_END_URL}/uploads/${this.formdata.imgname}`;
        console.log("Image URL: " + this.image);
      }
    });
}
```

- URL에서 num 파라미터를 추출
- this.image에 이미지 파일의 URL을 생성하여 호출

이미지 내 스팸 단어 검출과 등록 차단 기능을 가진 QnA 게시판

나의 문의 수정 하기

문의 수정 하기

문의 등록시 유의사항

(◉_◉_◉) 클린봇이 상업적인 사진, 욕설, 음란성 문구를 AI 기술로 감지하고 있습니다.

wjdgksmf

2024-08-27

회원관련

회원관련

회원관련

첨부파일

파일 선택

선택된 파일 없음

안녕하세요 오늘은 중요한 수업이 있습니다.

수정하기

목록으로



```
update() {
  const formData = new FormData();
  formData.append('qna', JSON.stringify(this.formdata));
  if (this.image) {
    formData.append('file', this.image);
  }

  axios.post(`${process.env.VUE_APP_BACKEND_URL}/qna/qnaupdate`, formData, {
    headers: {
      'Content-Type': 'multipart/form-data'
    }
  })
  .then(() => {
    alert('Q&A 상태 및 답변이 성공적으로 수정되었습니다.');
    this.$router.push('/Qnalist');
  })
  .catch(error => {
    alert('Q&A 상태 및 답변 수정 중 오류가 발생했습니다.');
  });
},
```

- 수정이 성공하면 .then 콜백함수를 실행해 수정성공 알람을 띄움
- 반대로 실패하면 .catch 콜백함수를 실행해 수정 실패를 띄움

S2 Chat

Thank You

A팀

2024 August 28